

SecureQwen: Leveraging LLMs for vulnerability detection in python codebases

Abdechakour Mechri^{a,b}, Mohamed Amine Ferrag^{c,b,*}, Merouane Debbah^d

^a École supérieure en informatique 08 Mai 1945 de Sidi Bel Abbès (ESI-SBA), Algeria

^b Technology Innovation Institute, 9639 Masdar City, Abu Dhabi, United Arab Emirates

^c Department of Computer Science, Guelma University, Guelma 24000, Algeria

^d Khalifa University of Science and Technology, P O Box 127788, Abu Dhabi, United Arab Emirates

ARTICLE INFO

Keywords:

Static analysis
Vulnerability detection
Codebase
Large language model
Software security
Security
Generative pre-trained transformers

ABSTRACT

Identifying vulnerabilities in software code is crucial for ensuring the security of modern systems. However, manual detection requires expert knowledge and is time-consuming, underscoring the need for automated techniques. In this paper, we present SecureQwen, a novel vulnerability detection tool leveraging large language models (LLMs) with a context length of 64K tokens to identify potential security threats in large-scale Python codebases. Utilizing a decoder-only transformer architecture, SecureQwen captures complex relationships between code tokens, enabling accurate classification of vulnerable code sequences across 14 common weakness enumerations (CWEs), including OS Command Injection, SQL Injection, Improper Check or Handling of Exceptional Conditions, Path Traversal, Broken or Risky Cryptographic Algorithm, Deserialization of Untrusted Data, and Cleartext Transmission of Sensitive Information. Therefore, we evaluate SecureQwen on a large Python dataset with over 1.875 million function-level code snippets from different sources, including GitHub repositories, Codeparrot's dataset, and synthetic data generated by GPT4-o. The experimental evaluation demonstrates high accuracy, with F1 scores ranging from 84% to 99%. The results indicate that SecureQwen effectively detects vulnerabilities in human-written and AI-generated code.

1. Introduction

In recent years, software systems' escalating complexity and interconnectedness have significantly increased the risk of security vulnerabilities (Jensen et al., 2024). Traditional methods of detecting these vulnerabilities, which predominantly rely on manual code reviews and expert analysis, are becoming increasingly inadequate due to the sheer volume and sophistication of modern software (Wu et al., 2024). The pressing need for more efficient and effective techniques has spurred the exploration and development of automated vulnerability detection methods. These methods employ a variety of approaches, including static and dynamic analysis, formal verification, symbolic execution, and, more recently, machine learning and deep learning techniques (Ahmad et al., 2024; Zhang et al., 2024a; Silva et al., 2023).

Static analysis tools examine the source code without executing it, identifying potential vulnerabilities by looking for patterns or signatures indicative of insecure coding practices (Ozturk et al., 2023; Cheng et al., 2021). Examples of such tools include linting tools and static analyzers such as Flawfinder, Clang, and CPPCheck. More advanced formal verification tools like ESBMC and CBMC use mathematical models to

prove the absence of specific types of errors (Menezes et al., 2024). On the other hand, dynamic analysis involves executing the code in a controlled environment to monitor its behavior and detect anomalies that could signify security flaws (Zhang et al., 2023). Tools like fuzz testers and runtime analyzers fall into this category (Senanayake et al., 2023). While these techniques can be powerful, they often struggle with scalability and detecting complex, context-dependent vulnerabilities (Xia et al., 2024).

As software complexity and susceptibility to vulnerabilities rise, the need for automated vulnerability detection becomes crucial yet challenging. With LLMs achieving remarkable success in various applications, there is increasing confidence in their effectiveness in detecting vulnerabilities. However, there remains a lack of quantitative insight into their potential in this area. To address this gap, Gao et al. (2023) present VulBench, a comprehensive vulnerability benchmark that consolidates high-quality data from diverse Capture-the-Flag (CTF) challenges and real-world applications. VulBench provides annotations for each vulnerable function, specifying the vulnerability type and its root cause. Experiments involving 16 LLMs and 6 state-of-the-art

* Corresponding author at: Department of Computer Science, Guelma University, Guelma 24000, Algeria.

E-mail addresses: a.mechri@esi-sba.dz (A. Mechri), ferrag.mohamedamine@univ-guelma.dz (M.A. Ferrag), merouane.debbah@ku.ac.ae (M. Debbah).

deep learning models and static analyzers demonstrate that several LLMs surpass traditional deep learning methods in vulnerability detection, highlighting their untapped potential. Another study (Mao et al., 2024) highlights the advancements in LLMs for vulnerability detection, emphasizing the need for diverse viewpoints from different roles in the software development lifecycle. This study introduces a multi-role approach, employing LLMs to simulate real-life code reviews and engage in discussions toward a consensus on vulnerabilities, resulting in significant improvements in precision, recall, and F1 scores. Despite progress in building secure systems, Android applications remain vulnerable, necessitating effective detection methods. Traditional static and dynamic analysis tools face limitations like high false positives and limited analysis scope. In addition, Mathews et al. (2024) explore LLMs' efficacy in detecting Android vulnerabilities, demonstrating that LLMs can effectively identify and rectify vulnerabilities. Experiments show that LLMs correctly flag insecure apps in 91.67% of cases using the Ghera benchmark, highlighting their potential to build a robust and actionable vulnerability detection system (Yusuf and Jiang, 2024).

Zhou et al. (2024) tackled the challenges of software vulnerability detection by undertaking a detailed analysis that contrasts Static Application Security Testing (SAST) tools with Large Language Models (LLMs) across various programming languages. They devised a new task that allows LLMs to detect vulnerabilities at the repository level, focusing on Java, C, and Python. To facilitate this investigation, they developed three specific datasets for each language, enabling an in-depth comparison of SAST tools and LLMs. The evaluation framework they employed includes 12 different LLMs, with a range of architectures from encoder-only to encoder-decoder models and parameter sizes spanning from 120 million to 8 billion. The study's key findings indicate that while SAST tools showed limited success, with vulnerability detection ratios peaking at 44.4%, LLMs achieved higher detection rates (between 90% and 100%) but also produced many false positives. Additionally, combining multiple SAST tools led to a marked improvement in detection rates (25.2%–100.0%), and combining diverse LLMs significantly reduced false positive rates (40.9%–74.6%). Significantly, the best approach varied depending on the programming language: combined LLMs were most effective for Java, whereas combined SAST tools yielded better results for C and Python.

In light of these advancements, machine learning, and deep learning have opened new avenues for vulnerability detection, offering the potential to automatically learn and generalize patterns from vast amounts of code data. Recent studies have demonstrated the efficacy of these techniques in identifying a wide range of vulnerabilities with high accuracy (Lin et al., 2020; Subhan et al., 2022). Among these, large language models (LLMs) have shown particular promise. These models, trained on extensive code datasets, can capture intricate relationships and semantic nuances within the code that traditional methods might miss. For instance, models like CodeBERT and GraphCodeBERT have been designed explicitly for code analysis and have shown significant improvements in tasks such as code completion and vulnerability detection (Shestov et al., 2024; Guo et al., 2020; Feng et al., 2020).

However, most LLM-based approaches proposed by researchers are designed for C/C++ codebases, mainly because of the availability of public datasets for these languages. There is a noticeable lack of public datasets for Python, which has limited the development of LLM-based vulnerability detection models for this language (Lu et al., 2024; Zhang et al., 2024b). To address this gap, we are exploring the creation of datasets for Python code at both the source code and function levels. Traditional tools like BMC-based models (Farias et al., 2024), Checkmarx, Bandit, PyLint, and Coverity, while helpful, can be time-consuming, prone to high false positive rates, and cannot frequently provide detailed explanations for the vulnerabilities they detect (Li et al., 2024). By leveraging the power of LLMs to classify vulnerabilities in Python codebases, we aim to enhance detection accuracy and provide detailed explanations for the vulnerabilities detected, facilitating effective remediation. This innovative approach not only advances the

field of automated vulnerability detection but also addresses the critical need for more robust and efficient security tools in the ever-evolving landscape of software development.

In this paper, we introduce SecureQwen, an advanced vulnerability detection tool that leverages a decoder-only transformer architecture to analyze Python code. SecureQwen is trained on a comprehensive dataset comprising over 1.875 million function-level code snippets from diverse sources, including GitHub repositories, Codeparrot's dataset, and synthetic data generated by advanced LLMs. This extensive training allows SecureQwen to accurately classify vulnerable code sequences across a range of common weakness enumerations (CWEs), significantly enhancing the security of Python applications.

The remainder of this paper is organized as follows: Section 2 provides a comprehensive overview of the relevant literature on software vulnerability detection. In Section 3, we discuss the methodology and approach used to develop SecureQwen. Section 4 presents the data collection. Section 5 describes the experimental setup and presents the evaluation of SecureQwen's performance. Finally, Section 6 concludes the paper by summarizing the key concepts, findings, contributions, and implications.

2. Related work

Software ecosystems are pivotal in modern development due to their open platforms of reusable packages, which expedite and simplify tasks. However, this reuse complicates the detection of security vulnerabilities, as systems rely on numerous packages. Alfadel et al. (2023) focuses on the Python ecosystem (PyPi), presenting an empirical study of 1396 vulnerability reports affecting 698 Python packages. The study examines the propagation and lifespan of these vulnerabilities, noting that they take over three years to be discovered and are often only fixed after public announcement, leaving significant time for exploitation. Additionally, an analysis of 2224 GitHub Python projects reveals that over half depend on at least one vulnerable package, with updates to secure versions taking an average of seven months. The findings highlight an increasing trend in Python package vulnerabilities and suggest improvements in the discovery, fixing, and management processes to enhance ecosystem security.

Wartschinski et al. (2022) proposed an automated tool for identifying vulnerable code using deep learning. Vudenc leverages a word2vec model to generate vector representations of semantically similar code tokens and employs a network of short-term memory (LSTM) cells to classify vulnerable code sequences at a fine-grained level. Trained on 1009 vulnerability-fixing commits from various GitHub repositories encompassing seven vulnerabilities, Vudenc achieves impressive recall, precision, and F1 scores ranging from 78% to 96%. The experimental results indicate that Vudenc surpasses most existing tools in vulnerability detection accuracy on real-world software, offering developers precise vulnerability identification with high confidence levels and minimal setup effort.

Antal et al. (2020) proposed a comprehensive analysis of software security in the context of open-source development, highlighting the persistent challenges in effectively addressing security issues. By examining a substantial dataset of commits from Python and JavaScript projects in public repositories using the Software Heritage Graph Dataset, they categorized vulnerabilities mitigated by developers and assessed their distribution and resolution times. Their study identified common vulnerabilities across both communities while noting language-specific prevalent issues. Furthermore, the authors observed that the JavaScript and Python communities generally respond slowly to newly identified security vulnerabilities, with few exceptions for certain vulnerability types. This research aims to enhance security management practices by providing insights into cybersecurity issues in open-source software development.

Zhou et al. (2021) proposed VulFixMiner, a Transformer-based approach designed to automatically detect silent vulnerability fixes in

Table 1

Comparative analysis of related work based on vulnerabilities, target languages, models, and dataset sources.

Study	Vulnerabilities	Target language(s)	Context window	Model/Tool used	Dataset source
Antal et al. (2020)	Common vulnerabilities in open-source projects	Python, JavaScript	N/A	Software Heritage Graph Dataset	Commits from Python and JavaScript projects
Bhandari et al. (2021)	CVE-recorded vulnerabilities	Various	N/A	CVEfixes tool	National Vulnerability Database (NVD) and associated open-source repositories
Zhou et al. (2021)	Silent vulnerability fixes	Java, Python	512	Transformer-based model	Sampled commits from 204 projects
Wartschinski et al. (2022)	Specific vulnerabilities (7 types)	Python	N/A	LSTM-based model with word2vec	1009 vulnerability-fixing commits from GitHub
Mamede et al. (2022)	21 types of vulnerabilities in Java files	Java	512	Transformer-based model integrated into Visual Studio Code	Java files from various stages of the SDLC
Alfadel et al. (2023)	General Python package vulnerabilities	Python	N/A	Empirical analysis	PyPi and GitHub projects
Eniser et al. (2024)	Translation-related issues in Rust	Rust	N/A	Differential fuzzing with LLMs	Code from open-source projects
Farasat and Posegga (2024)	7 Vulnerabilities	Python	N/A	Bidirectional Long Short-Term Memory (BiLSTM) model	1009 vulnerability-fixing commits from GitHub
Ferrag et al. (2024)	25 most dangerous software weaknesses	C/C++	2k	Decoder-only transformer architecture	A combination of public datasets, namely the SySeVR framework, Draper VDISC, Bigvul, Diversevul, SARD Juliet, and ReVeal datasets
Mohammed et al. (2024)	Memory safety violations in C	C, CheckedC	N/A	LLMs with lightweight static analysis in MSA tool	Benchmarks and real-world code (up to 20,000 lines)
SecureQwen (Proposed Model)	14 CWEs	Python	64k	Decoder-only transformer architecture	1.875 million function-level code snippets from GitHub and LLM generated synthetic data

open-source software (OSS). Unlike the traditional coordinated vulnerability disclosure model, which delays the public disclosure of vulnerability fixes, VulFixMiner allows OSS users to identify these fixes promptly, reducing the risk of exploitation by malicious actors who probe for vulnerabilities once a fix is pushed to the source code repository. The model leverages semantic extraction from commit-level code changes, trained on sampled commits from 204 projects and evaluated on 52 additional projects. VulFixMiner demonstrates superior performance over state-of-the-art baselines, achieving AUC scores of 0.81 for Java and 0.73 for Python datasets, and effectively identifying 49% of total vulnerability fixes by inspecting just 5% of the total lines of code. Manual verification further validates its efficacy, revealing that 35% of unmarked sampled commits are indeed fixed for vulnerabilities.

[Bhandari et al. \(2021\)](#) introduced a method to automatically gather and organize a comprehensive dataset of security vulnerabilities and their fixes from Common Vulnerabilities and Exposures (CVE) records found in the National Vulnerability Database (NVD). This method, implemented in a tool called CVEfixes, automatically collects CVE records, retrieves the associated vulnerable code and fixes from open-source repositories, and stores this information in a relational database enriched with metadata, including programming language and detailed code and security metrics across five levels of abstraction. The initial release of CVEfixes, encompassing data up to June 9, 2021, includes 5365 CVE records from 1754 open-source projects and 5495 vulnerability fixing commits. This dataset supports various forms of data-driven software security research, such as vulnerability prediction, classification, severity prediction, analysis of code changes related to vulnerabilities, and automated vulnerability repair. Therefore, [Hashemi Chaleshtori and Ray \(2022\)](#) proposed an approach to vulnerability information extraction using Transformer models like BERT, XLNet, RoBERTa, and

DistilBERT, which have demonstrated superior performance in NLP tasks such as Named Entity Recognition (NER) and Co-reference Resolution. By fine-tuning these models on a labeled dataset from vulnerability databases, they aim to automate the extraction of security-related terms from vulnerability descriptions. This method surpasses traditional rule-based and CRF-based models by eliminating the need for manual feature selection and expert domain knowledge. It effectively identifies new information from atypical vulnerability descriptions, enhancing the timeliness and accuracy of vulnerability mitigation.

[Mamede et al. \(2022\)](#) presented VDet for Java, a transformer-based extension for Visual Studio Code, designed to enhance code security by enabling the early detection of vulnerabilities in Java files within the software development life cycle (SDLC). Leveraging the powerful Transformers architecture, known for its advancements in deep learning for Natural Language Processing, VDet for Java aims to address the gap between state-of-the-art deep learning techniques and their application in development environments. The preliminary evaluation of their model demonstrates an impressive 98.9% accuracy in multi-label classification, capable of identifying up to 21 types of vulnerabilities.

[Eniser et al. \(2024\)](#) proposed FLOURINE, an end-to-end code translation tool utilizing differential fuzzing to assess if Rust translations are I/O equivalent to their source programs. This study marks the first substantial evaluation of large language models (LLMs) in translating real-world code to Rust. They examine five state-of-the-art LLMs: GPT-4, Claude 3, Claude 2.1, Gemini Pro, and Mixtral. Conducting their analysis of code from open-source projects, the authors evaluate the initial success of translations and the LLMs' ability to rectify buggy translations. Automated feedback strategies, including counterexamples, are used when translations are not I/O equivalent. The findings

reveal that the most effective LLM can accurately translate 47% of the benchmarks, providing valuable insights for future improvements.

Mohammed et al. (2024) proposed an approach to address memory safety violations in low-level code, particularly C, by porting it to a safe C dialect. This process, which typically requires extensive manual effort for annotation and code refactoring, is enhanced LLMs. The authors introduce a framework that combines LLMs with lightweight static analysis to facilitate complex code reasoning and rewriting. This framework, implemented in the MSA tool targeting the CheckedC dialect, decomposes the LLM transformation process into manageable steps. Evaluation on various benchmarks and real-world code up to 20,000 lines demonstrates MSA's superior performance over a vanilla LLM baseline and a state-of-the-art symbolic technique, highlighting its effectiveness in enhancing memory safety with minimal programmer burden.

Table 1 presents a comparative analysis of related works based on vulnerabilities, target languages, models, and dataset sources, highlighting the unique contributions of our proposed model, SecureQwen. The key differentiator lies in its innovative approach to leveraging LLMs with an extended context length of 64k tokens, which allows it to process and analyze large-scale Python codebases effectively. Unlike existing tools, SecureQwen employs a decoder-only transformer architecture, enhancing its capability to capture complex relationships between code tokens. This architecture facilitates the accurate classification of vulnerable code sequences and extends its efficacy across 14 CWEs, including sophisticated threats like OS Command Injection and SQL Injection. Furthermore, SecureQwen's evaluation of a substantial dataset comprising over 1.875 million function-level code snippets from diverse sources, including GitHub repositories and synthetic data generated by GPT-4, showcases its robustness and versatility. The tool's impressive F1 scores, ranging from 84% to 99%, underline its superior performance in detecting vulnerabilities in both human-written and AI-generated code, setting a new benchmark in vulnerability detection and emphasizing its potential to enhance software security practices.

3. Model architecture

3.1. CodeQwen model

CodeQwen1.5-7B is a large language model developed by the Qwen team (Bai et al., 2023b), based on the Qwen1.5 open-source family. It is a decoder-only transformer-based model trained on 3 trillion tokens of code data. The model supports 92 programming languages and demonstrates strong code-generation capabilities. Additionally, it supports long-context understanding and generation with a maximum context length of 64K tokens and includes group query attention (GQA) for efficient inference. It has been evaluated on various benchmarks, showing excellent performance in text-to-SQL, bug fixing, and other coding tasks. As shown in Table 2, CodeQwen1.5-7B, despite having only 7 billion parameters, has outperformed larger models in basic code generation tasks. This achievement further reduces the proficiency gap between GPT-4 (OpenAI et al., 2023) and open-source code LLMs.

The Qwen architecture exemplifies an advanced methodology for constructing large language models, integrating recent developments in Transformer-based architectures. It builds upon the foundational work established by the LLaMA model (Touvron et al., 2023), a well-regarded open-source framework, by introducing several critical enhancements that improve both performance and computational efficiency.

- **Embedding and output mapping:** One key innovation in Qwen is using untied embeddings for both input and output layers. This approach diverges from the traditional method, which uses the same weights for these layers. By separating the weights, Qwen achieves improved performance metrics, though this comes at the cost of increased memory usage (Alammar, 2018).

Table 2

Performance comparison of CodeQwen1.5-7B with other models on various coding benchmarks.

Model	Size	HumanEval 0-shot	HumanEval+ 0-shot
Base model			
CodeLlama-Base	7B	33.5	25.6
StarCoder2	7B	35.4	29.9
DeepSeek-Coder-Base	6.7B	47.6	39.6
CodeQwen1.5	7B	51.8	45.7
Chat model			
GPT-3.5-Turbo	–	76.8	70.7
GPT-4-Turbo (Nov 2023)	–	85.4	81.7
DeepSeek-Coder-Instruct	6.7B	78.6	70.1
CodeQwen1.5-Chat	7B	83.5	78.7

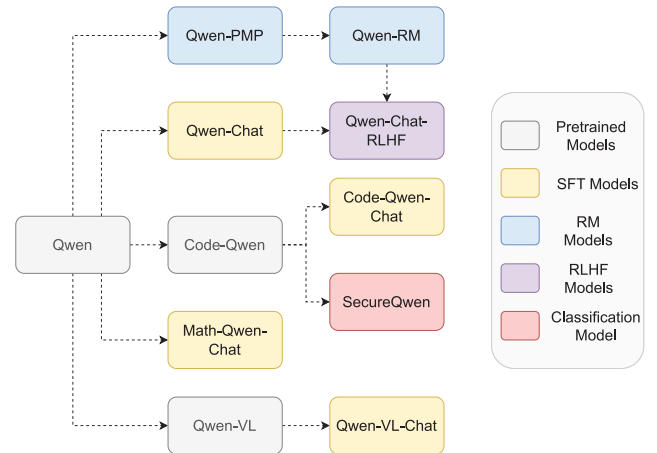


Fig. 1. Model lineage of the Qwen series compared to SecureQwen.

- **Positional embedding:** To encode positional information, Qwen utilizes Rotary Positional Embedding (RoPE) (Su et al., 2021). This technique has gained traction in other contemporary models, such as PaLM and LLaMA, for its effectiveness (Chowdhery et al., 2023). Qwen employs FP32 precision for the inverse frequency matrix within RoPE to maximize accuracy, prioritizing model performance over the memory efficiency offered by lower precisions like BF16 or FP16.
- **Bias:** Bias handling in Qwen reflects a thoughtful balance between model complexity and performance. While biases are removed from most layers to streamline the architecture, they are retained in the attention mechanism's Query-Key-Value (QKV) layers. This selective inclusion enhances the model's ability to generalize and extrapolate effectively.
- **Pre-normalization:** The model adopts pre-normalization techniques, which are preferred in modern Transformer designs for their ability to stabilize training. Additionally, Qwen replaces the conventional layer normalization with RMSNorm, which maintains performance while improving efficiency (Jiang et al., 2024; Bai et al., 2022).
- **Activation function:** Another significant aspect of Qwen is its activation function. The model employs SwiGLU, a combination of Swish and Gated Linear Unit (GLU), which has demonstrated superior performance in preliminary experiments compared to other activation functions like GeLU. Furthermore, Qwen optimizes the feed-forward network (FFN) dimension, reducing it to approximately 8/3 of the hidden size, a modification aimed at enhancing computational efficiency without compromising model capability (Shazeer, 2020).

Fig. 1 presents the model lineage of the Qwen series compared to SecureQwen. The authors of Qwen proposed a comprehensive model lineage for the Qwen series. They pre-trained the Qwen language models on vast datasets containing trillions of tokens. Following pre-training, they utilized Supervised Fine-Tuning (SFT) and Reinforcement Learning with Human Feedback (RLHF) to align Qwen with human preferences, leading to the development of Qwen-Chat and its enhanced version, Qwen-Chat-RLHF. Using similar techniques, the authors also created specialized coding and mathematics models, such as Code-Qwen, Code-Qwen-Chat, and Math-Qwen-Chat. They also previously released the multimodal language models Qwen-VL and Qwen-VLChat (Bai et al., 2023a), which are based on the Qwen base models. Compared to these different Qwen versions, we introduce SecureQwen, a cutting-edge vulnerability detection tool fine-tuned from CodeQwen specifically for classification tasks. SecureQwen utilizes the large language model with an extended context length of 64K tokens to effectively identify security threats in large-scale Python codebases. It employs a decoder-only transformer architecture that captures intricate relationships between code tokens, allowing for precise classification of vulnerable code sequences across 14 CWEs, such as OS Command Injection, SQL Injection, Improper Handling of Exceptional Conditions, Path Traversal, Broken or Risky Cryptographic Algorithms, Deserialization of Untrusted Data, and Cleartext Transmission of Sensitive Information.

3.2. SecureQwen model architecture

Fig. 2 presents the *SecureQwen* model architecture. The classification model comprises three main parts: the embedding layer, the decoder layers, and the classification head. The embedding layer converts input tokens into high-dimensional vectors. The decoder layers process these embeddings through a series of attention and feed-forward operations. Finally, the classification head maps the processed features to the output classes.

3.2.1. Embedding layer

The embedding layer is the initial component of the model, responsible for converting raw input tokens into dense, high-dimensional vectors (Mikolov et al., 2013). This transformation is essential as it captures the semantic meaning of the tokens, enabling the subsequent layers to process the information more effectively. Specifically, the embedding layer maps each input token into a 4096-dimensional space. The vocabulary size for this model is 92,416 tokens, meaning the embedding layer can handle a vast range of distinct input tokens.

For an input token x_i , the embedding layer produces a vector $e_i \in \mathbb{R}^{4096}$:

$$E(x_i) = W_e \cdot \text{one_hot}(x_i) \quad (1)$$

where $W_e \in \mathbb{R}^{92416 \times 4096}$ is the embedding matrix and $\text{one_hot}(x_i)$ is the one-hot encoded vector of the input token x_i .

3.2.2. Decoder layers

The model consists of 32 Decoder Layer instances, which are integral to the model's architecture. These layers process embedded tokens through multiple self-attention and feed-forward operations stages, enabling the model to capture complex dependencies in input sequences (Vaswani et al., 2017). Each Decoder Layer contains the following components:

Self-Attention Mechanism: The self-attention mechanism within each Decoder Layer involves several key steps:

- **Normalization Layers:** Normalization layers are used to stabilize the training process by normalizing the inputs of each layer. This helps in accelerating convergence and improving model performance (Su et al., 2021).

$$\text{RMSNorm}(X) = \frac{X}{\sqrt{\text{mean}(X^2) + \epsilon}} \quad (2)$$

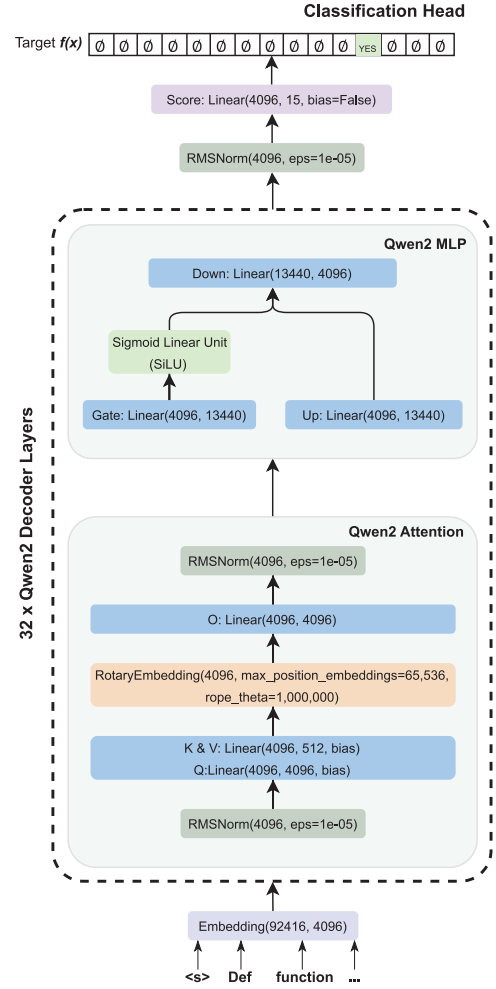


Fig. 2. *SecureQwen* model architecture.

- **Query, Key, Value Projections:** Input features are transformed through linear projections to produce the query (Q), key (K), and value (V) matrices:

$$Q = XW_Q \quad (3)$$

$$K = XW_K \quad (4)$$

$$V = XW_V \quad (5)$$

Here, $X \in \mathbb{R}^{n \times 4096}$ is the input matrix, $W_Q \in \mathbb{R}^{4096 \times 4096}$ and $W_K, W_V \in \mathbb{R}^{4096 \times 512}$ are the projection matrices (Vaswani et al., 2017).

- **Rotary Positional Embeddings:** The query and key matrices are augmented with rotary positional embeddings (RoPE) (Su et al., 2021), which encode positional information directly into the query and key vectors to enhance the model's ability to capture positional dependencies:

$$Q = \text{RoPE}(Q) \quad (6)$$

$$K = \text{RoPE}(K) \quad (7)$$

Table 3
Data sources for code Snippets.

Source	Description	Size
GitHub Scraping	Scraped using targeted keywords.	43 k
Codeparrot's dataset	Curated Python code snippets ^a .	1190 k
Synthetic Data	Generated using different LLMs Gpt 3.5, CodeQwen and CodeLlama	640 k
Total		1875 k

^a <https://huggingface.co/datasets/codeparrot/github-code-clean>.

The rotary embeddings are applied as part of the attention mechanism, enabling more effective learning of positional relationships.

- **Scaled Dot-Product Attention:** The attention scores are computed as follows:

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \quad (8)$$

where d_k is the dimension of the key vectors.

- **Output Projection:** The final output of the self-attention mechanism is obtained by projecting the attention result:

$$\mathbf{O} = \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V})\mathbf{W}_O \quad (9)$$

where $\mathbf{W}_O \in \mathbb{R}^{4096 \times 4096}$ is the output projection matrix.

Multi-Layer Perceptron (MLP): The MLP (Popescu et al., 2009) layer applies additional transformations to the features, enabling the model to learn more complex representations. This layer consists of multiple linear transformations and a non-linear activation function.

- **Activation Function:** SiLU (Sigmoid Linear Unit) (Elfwing et al., 2017) for non-linearity:

$$\text{SiLU}(x) = x \cdot \text{sigmoid}(x) \quad (10)$$

- **Gate, Up and Down Projections:** Implement the MLP with intermediate dimension expansion and contraction.

$$\mathbf{H}_1 = \text{SiLU}(\mathbf{X}\mathbf{W}_{gate}) \quad (11)$$

$$\mathbf{H}_2 = \mathbf{H}_1 \odot \mathbf{X}\mathbf{W}_{up} \quad (12)$$

$$\text{Out} = \mathbf{H}_2\mathbf{W}_{down} \quad (13)$$

where $\mathbf{W}_{gate}, \mathbf{W}_{up} \in \mathbb{R}^{13440 \times 4096}$ and $\mathbf{W}_{down} \in \mathbb{R}^{13440 \times 4096}$.

3.2.3. Classification head

The classification head maps the processed features to the final output classes, one of the 15 CWEs. This is done using a linear layer, which produces a score for each class.

$$\mathbf{Y} = \mathbf{X}\mathbf{W}_{score} \quad (14)$$

where $\mathbf{W}_{score} \in \mathbb{R}^{4096 \times 15}$ is the weight matrix of the classification head. This transformation ensures that the processed data is mapped to the correct output class, completing the classification task.

4. Datasets for Fine-tuning SecureQwen

4.1. Data collection

Diverse and comprehensive data collection is essential for effectively training and evaluating LLMs (Zhao et al., 2023). In this study, we employed three primary sources of data (Fig. 4): GitHub scraping, open-source datasets, and synthetic data generation using LLMs as shown in data sources, as presented in Table 3.

4.1.1. GitHub scraping

GitHub,¹ a widely-used code hosting platform, offers an extensive repository of open-source code (Kalliamvakou et al., 2014). To extract relevant code snippets, we adopted a targeted approach. Keywords such as “security”, “vulnerability”, “exploit”, and “patch” were used to identify repositories likely to contain pertinent code. We further applied specific filtering conditions to ensure the quality and relevance of the collected data. These conditions included focusing on Python file types, targeting active repositories through commit history analysis, and considering code length. This methodology aimed to gather code snippets that are both relevant to security and representative of real-world programming practices.

4.1.2. Codeparrot's Github-code-clean dataset

In addition to scraping GitHub, we utilized the Codeparrot dataset,² which contains high-quality, curated Python code snippets. We extracted 1.19 million function-level code snippets from this dataset, ranging from 3 to 200 lines. This provided us with a diverse set of clean and well-prepared code examples.

4.1.3. Synthetic data generation with LLMs

To further enhance diversity and generalizability, synthetic code snippets were generated using LLMs, specifically ChatGPT-3.5 Turbo, GPT4-o (OpenAI, 2023), CodeQwen1.5-7B-Chat (Bai et al., 2023b), and CodeLlama-7b-hf (Rozière et al., 2024). These examples were crafted based on various use cases, exposing the LLMs to a broader spectrum of coding patterns. Incorporating synthetic data significantly improved the LLM's generalization ability to new and unseen code. In total, approximately 640,000 synthetic source code snippets were generated. We created a comprehensive and diverse experimental dataset by combining real-world, curated, and synthetic data. This rich collection of code snippets was crucial for conducting experiments, encompassing various coding examples and practices. This diversity provided a robust foundation for enhancing the training process, ensuring better performance and adaptability of the LLMs.

- The algorithm 1 for generating safe Python code utilizes a language model (LLM) to create diverse and compilable code snippets. It begins by initializing a list to store the generated safe code snippets. A prompt template is defined, which instructs the LLM to generate code that follows best practices, avoids security vulnerabilities, and ensures the program compiles successfully. The algorithm iterates over a predefined list of types (e.g., Wi-Fi Signal Strength Analyzer, QR Code Reader, Image Steganography) and styles (e.g., excited, relaxed, mathematical). For each combination of type and style, the LLM generates a safe code snippet. The generated code is then tested for compilability. If the code compiles successfully, it is saved to a .py file, and its metadata (type, style, file name) is appended to the list of safe code snippets. If the code does not compile, it is discarded. Finally, the metadata of all successful safe code snippets is saved to a JSON file, providing a structured dataset of safe Python code.
- The algorithm 2 for generating vulnerable Python code follows a similar structure to the safe code generation process but introduces additional steps to incorporate security vulnerabilities. An empty list is initialized to store the vulnerable code snippets. A

¹ <https://github.com/>.

² <https://huggingface.co/datasets/codeparrot/github-code-clean>.

Algorithm 1 Generating Safe Python Code**Require:** LLM, code types, code styles**Ensure:** Synthetic dataset containing safe Python code

```

1: Initialize list for storing safe code snippets:
2: safe_code_snippets ← empty list
3: Define prompt template:
4: safe_code_prompt_template ← "Generate a safe Python code snippet. The code should follow best practices and not contain any security vulnerabilities. Type: {type}. Style: {style}."
5: Define types and styles:
6: types ← ["Wi-Fi Signal Strength Analyzer", "QR Code Reader", "Image Steganography", "Pixel Art Generator", "Scientific Calculator Implementation", "Weather Data Fetcher", "Text Encryption and Decryption", "File Compression Tool", "Chatbot Application", "Currency Converter", "Web Scraper", "To-Do List Application", "Expense Tracker", "Stock Price Analyzer", "Basic Game", "Email Sender", "Data Visualizer", "API Request Handler", "Password Generator", "Sorting Algorithm Implementations"]
7: styles ← ["mathematical", "happy", "functional", "object-oriented", "script-like", "modular", "beginner-friendly", "expert-level", "educational", "professional", "playful", "serious"]
8: for each type in types do
9:   for each style in styles do
10:    safe_code_prompt ← safe_code_prompt_template with type and style
11:    safe_code ← LLM.generate(safe_code_prompt)
12:    if safe_code is compilable then
13:      file_name ← "safe_code_" + hash(type + style) + ".py"
14:      safe_code_snippets.append({type, style, file_name, "safe"})
15:    else
16:      DISCARD generated safe code
17:    end if
18:  end for
19: end for
20: Save safe code snippets to a JSON file

```

Algorithm 2 Generating Vulnerable Python Code**Require:** LLM, vulnerability descriptions, code types, code styles**Ensure:** Synthetic dataset containing vulnerable Python code

```

1: Initialize list for storing vulnerable code snippets:
2: vulnerable_code_snippets ← empty list
3: Define prompt template:
4: vulnerable_code_prompt_template ← "Generate a vulnerable Python code snippet. The code should contain {vulnerability_type} vulnerabilities. Type: {type}. Style: {style}."
5: Define types and styles:
6: types ← ["Wi-Fi Signal Strength Analyzer", ...etc.]
7: styles ← ["mathematical", ...etc.]
8: vulnerability_types ← ["SQL Injection", "Cross-Site Scripting (XSS)", "Buffer Overflow", "Insecure Deserialization", "Path Traversal", "Command Injection", "Insecure Randomness", "Hardcoded Credentials", "Missing Authentication", "Insecure Cryptographic Storage", ...etc.]
9: for each type in types do
10:  for each style in styles do
11:    for each vulnerability_type in vulnerability_types do
12:      vulnerable_code_prompt ← vulnerable_code_prompt_template with type, style, and vulnerability_type
13:      vulnerable_code ← LLM.generate(vulnerable_code_prompt)
14:      if vulnerable_code is compilable then
15:        file_name ← "vulnerable_code_" + hash(type + style + vulnerability_type) + ".py"
16:        vulnerable_code_snippets.append({type, style, file_name, vulnerability_type})
17:      else
18:        DISCARD generated vulnerable code
19:      end if
20:    end for
21:  end for
22: end for
23: Save vulnerable code snippets to a JSON file

```

prompt template instructs the LLM to include specific vulnerabilities within the generated code, such as SQL Injection, Cross-Site Scripting, and Buffer Overflow. We employed these algorithms to ensure our dataset includes safe and vulnerable code snippets.

4.2. Data labeling

Accurate labeling of vulnerabilities within source code is essential for developing security analysis tools. We utilized multiple static analysis tools to identify common security issues in Python code, including

Bandit, Semgrep, SonarQube, CodeQL, and PyLint. These tools process each file by constructing an Abstract Syntax Tree (AST) and running appropriate analyses to detect security vulnerabilities. The AST is a data structure that represents the syntax or structure of a program, focusing on structural details and omitting syntax specifics like grouping parentheses (Alikhanifard and Tsantalis, 2024; Finamore, 2024). Once a file is processed, the tools run the relevant checks against the AST nodes and generate reports outlining potential security issues.

In addition to the static analysis tools, we leveraged several leading LLM models, including GPT-4, CodeWhisperer, DeepSeek-Coder, and CodeLlama, to detect and label vulnerabilities. This multi-faceted approach was designed to enhance the accuracy and robustness of the dataset by utilizing the complementary strengths of these tools. We conducted a manual review of 100 randomly selected samples to ensure the accuracy of the annotations. This step was undertaken to cross-verify the automated labels and assess the tools' reliability.

An illustration of this process is shown in Fig. 3. In the first example, Bandit identified a potential SQL injection vulnerability in the Python function *find_user*. It flagged the direct use of the username variable in the SQL query and provided relevant links to the Common Weakness Enumeration (CWE³) entry for additional information. Another high-severity issue detected by Bandit involved a subprocess call in the function *list_directory*. The use of *subprocess.Popen* with *shell = True* was flagged as a potential command injection vulnerability since user input is directly passed to the shell, allowing an attacker to inject and execute arbitrary commands. This vulnerability is linked to CWE-78, and in the example provided, simulated malicious user input could lead to destructive actions like deleting critical system files. Additionally, Bandit detected a cryptographic weakness in the function *create_weak_dsa_key*, where a DSA key of 512 bits was generated. This key size is considered insecure by modern standards, as it can be easily broken by attackers, compromising the confidentiality and integrity of the encrypted data. This vulnerability is associated with CWE-326.

4.3. Analysis of static analysis tools for dataset labeling

Accurate labeling of vulnerabilities within source code is essential for developing practical security analysis tools. In this subsection, we present an analysis of the detection capabilities of various static analysis tools employed for labeling our Python code dataset. By leveraging multiple tools, we aim to enhance the robustness and accuracy of the labeling process through cross-verification of detected vulnerabilities. Table 4 summarizes the counts of each CWE instance in our dataset alongside the detection capabilities of the static analysis tools used: Bandit, Semgrep, SonarQube, CodeQL, and PyLint.

We employed multiple static analysis tools to label our dataset effectively, each contributing unique strengths to detecting vulnerabilities in Python source code. Bandit, being a Python-specific security analyzer, provided reliable detection for a majority of the critical CWEs, such as CWE-22 (Path Traversal), CWE-78 (OS Command Injection), and CWE-89 (SQL Injection). Its focus on Python and ease of integration into development workflows made it an essential tool in our labeling process. Semgrep and SonarQube complemented Bandit's capabilities by offering customizable rules and comprehensive code analysis, which helped detect vulnerabilities that Bandit partially covered or missed. In addition, CodeQL was utilized for its powerful semantic code analysis through custom queries, allowing us to identify complex vulnerabilities that other tools might not detect readily. PyLint contributed by enforcing coding standards and detecting certain security issues, such as CWE-259 (Hard-coded Password). The combined use of these tools ensured a more comprehensive coverage of CWEs in our dataset and enhanced the accuracy of our labeling through cross-verification of detected vulnerabilities.

³ <https://cwe.mitre.org/>.

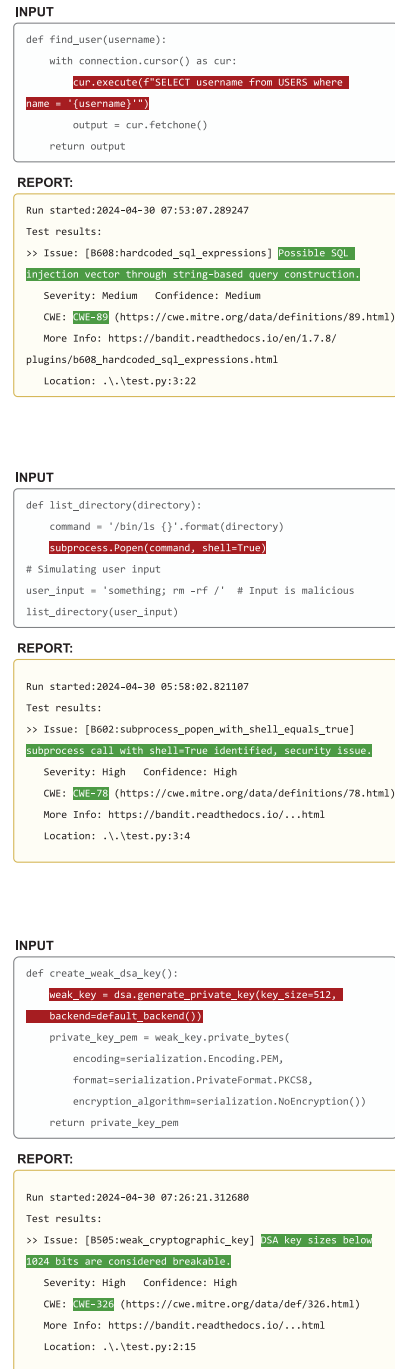


Fig. 3. Sql Injection, Os command Injection and Weak Cryptographic Key vulnerabilities reported by bandit static analyzer.

The final dataset, PythonVulnDB, is summarized in Table 4. It comprises approximately 1.875 million function-level samples categorized under more than 15 labels. While most functions were assigned a single label, others had multiple labels. We found that selecting the most frequent CWE label in these cases provided the best results. Additionally, the dataset includes samples labeled as UNK, which stands for unknown. These samples were poorly formatted and could not be converted to an Abstract Syntax Tree (AST) by the static analyzer, leaving their vulnerability status indeterminate. The PythonVulnDB dataset offers a substantial number of samples per CWE category, ensuring diversity and volume. This extensive and varied dataset will enable robust training for vulnerability detection models. Despite the inclusion

Table 4
CWE counts and detection capability by various tools.

CWE ID	Description	Count	Bandit	Semgrep	SonarQube	CodeQL	PyLint
SAFE	Safe From Any Vulnerability	1580K	N/A	N/A	N/A	N/A	N/A
UNK	Unknown	332K	N/A	N/A	N/A	N/A	N/A
CWE-703	Improper Check or Handling of Exceptional Conditions	80K	Yes	Partial	Yes	Yes	Partial
CWE-400	Uncontrolled Resource Consumption	40K	Partial	Partial	Yes	Partial	No
CWE-22	Path Traversal	27K	Yes	Yes	Yes	Yes	Partial
CWE-78	OS Command Injection	25K	Yes	Yes	Yes	Yes	Partial
CWE-327	Broken or Risky Cryptographic Algorithm	18K	Yes	Yes	Yes	Yes	No
CWE-330	Insufficiently Random Values	16K	Yes	Yes	Yes	Yes	No
CWE-605	Multiple Binds to the Same Port	16K	Partial	No	Partial	Partial	No
CWE-89	SQL Injection	15K	Yes	Yes	Yes	Yes	Partial
CWE-502	Deserialization of Untrusted Data	14K	Partial	Partial	Yes	Yes	No
CWE-259	Hard-coded Password	13K	Yes	Yes	Yes	Yes	Yes
CWE-20	Improper Input Validation	11K	Partial	Partial	Yes	Yes	Partial
CWE-319	Cleartext Transmission of Sensitive Information	9K	Yes	Yes	Yes	Yes	No
CWE-377	Insecure Temporary File	9K	Yes	Yes	Yes	Yes	No
CWE-295	Improper Certificate Validation	1K	Partial	Partial	Yes	Yes	No

Yes: The tool can reliably detect this CWE; Partial: The tool can detect some instances of this CWE but may not cover all cases; No: The tool does not typically detect this CWE; N/A: Not applicable for SAFE and UNK categories.

Table 5
Function-level training dataset.

Label	Tokens	Mean	Std	Min	Max	Count
SAFE	2448k	90.35	84.89	6	2048	27,102
CWE-703	1227k	136.91	110.62	9	2048	8,965
CWE-78	967k	107.58	72.56	11	1519	8,991
CWE-605	967k	107.13	56.38	19	910	9,027
CWE-89	901k	100.54	85.77	23	2048	8,966
CWE-259	995k	110.95	84.13	13	1771	8,975
CWE-22	879k	97.94	48.39	20	440	8,982
CWE-330	838k	93.51	66.41	10	2048	8,971
CWE-377	820k	118.46	82.47	11	1036	6,924
CWE-400	794k	88.21	47.55	16	670	9,007
CWE-327	740k	82.77	49.58	18	1126	8,952
CWE-319	754k	98.82	39.29	19	392	7,640
CWE-20	766k	84.98	51.06	16	1183	9,023
CWE-502	678k	75.16	50.48	17	956	9,027
CWE-295	107k	105.08	53.73	21	512	1,022

Table 6
Function-level test dataset.

Label	Tokens	Mean	Std	Min	Max	Count
SAFE	263k	90.78	81.64	8	855	2898
CWE-703	135k	130.24	95.36	12	1400	1035
CWE-605	105k	108.27	56.22	20	485	973
CWE-259	113k	109.99	74.80	15	604	1025
CWE-78	111k	110.22	72.06	11	569	1009
CWE-330	100k	97.21	86.16	12	2048	1029
CWE-327	88k	83.73	48.03	19	339	1048
CWE-89	97k	94.18	61.51	25	628	1034
CWE-22	99k	97.45	48.08	22	364	1018
CWE-377	88k	119.52	99.89	17	1642	740
CWE-400	84k	84.80	42.54	17	353	993
CWE-319	86k	98.50	41.63	18	282	869
CWE-20	81k	82.85	49.52	21	552	977
CWE-502	69k	71.12	46.83	18	584	973
CWE-295	12k	111.61	58.12	28	376	110

of unknown samples, the overall quality and comprehensiveness of the dataset make it a valuable resource for advancing the accuracy and reliability of our training processes.

While Bandit emerged as the most effective tool for detecting the prevalent CWEs in our Python codebase, the complementary use of Semgrep, SonarQube, CodeQL, and PyLint was essential to address the limitations of any single tool. This multi-tool approach ensured a robust labeling process, providing a solid foundation for subsequent vulnerability detection and analysis.

5. Performance evaluation

5.1. Experimental setup

In our experimental setup, we aimed to create a balanced and representative dataset for our analysis. To achieve this, we selected classes with a minimum of 1000 unique samples to ensure sufficient representation across different categories. Each class was then limited to a maximum of 10,000 samples to maintain balance, except the “SAFE” class, which was capped at 30,000 samples. This careful selection and curation process resulted in the following final distribution of classes in the dataset.

The training and test dataset statistics are summarized in [Tables 5 & 6](#). The “Tokens” column represents the total number of tokens in that label, while the “Mean”, “Std”, “Min”, and “Max” columns denote the mean, standard deviation, minimum, and maximum number of tokens per sample within each label, respectively. The “Count” column indicates the number of samples (functions) in each label.

The configuration of SecureQwen, detailed in [Table 7](#), employs the CodeQwen1.5-7B pretrained model as the foundation for the experiment. The model architecture is characterized by a hidden size of 4096, which determines the dimensionality of the vector space in which the model operates. Furthermore, the intermediate size of the multi-layer perceptron (MLP) is set to 13,440, influencing the complexity of the transformations applied to the input data. Additionally, the maximum number of position embeddings is 65,536, allowing the model to handle input sequences of considerable length.

The model type is qwen2, a variant of the transformer architecture built by the Qwen Team ([Bai et al., 2023b](#)). This architecture comprises 32 decoder layers, each of which applies self-attention and feed-forward neural network (FFNN) transformations to the input data. Notably, the number of key-value heads is set to 4, determining the number of parallel attention mechanisms employed by the model.

The RMS norm epsilon is set to $1e-05$, a small value used to stabilize the RMS normalization process ([Zhang and Sennrich, 2019](#)). Moreover, the RoPE theta is set to 1,000,000, a hyperparameter specific to the RoPE (Rotary Position Embedding) ([Su et al., 2021](#)) method used for encoding positional information. The Adam optimizer ([Kingma and Ba, 2017](#)) is used with specific betas (0.9 and 0.999) and epsilon ($1e-08$), controlling the adaptation of the learning rate during training.

We utilized the bfloat16 data type, a lower-precision floating-point format that can reduce memory usage and improve computational efficiency ([Xie et al., 2022](#)). Moreover, caching is enabled to speed up the computation by reusing previously computed results. The vocabulary size is 92,416, determining the number of unique tokens the model can process. The learning rate is set to 0.0001, controlling the step size of each optimization step. The training and evaluation batch sizes

Table 7
Configuration of SecureQwen.

Parameter	Value
Pretrained model ID	CodeQwen1.5-7B
BOS token ID	2
EOS token ID	2
Hidden activation	SiLU
Hidden size	4096
Intermediate size (MLP)	13,440
Max position embeddings	65,536
Model type	qwen2
Number of decoder layers	32
Number of key value heads	4
RMS norm epsilon	1e-05
RoPE theta	1,000,000
Torch Dtype	bfloat16
Use cache	True
Vocabulary size	92,416
Learning rate	0.0001
Number of classification labels	15
Train batch size	4
Eval batch size	8
Seed	42
Distributed type	Multi-GPU
Total train batch size	32
Total eval batch size	64
Optimizer	Adam with betas = (0.9, 0.999), epsilon = 1e-08
Number of epochs	1
GPU	16x A100 40 GB
Dataset split (Function)	90/10
Dataset split (Source Code)	80/20
Framework versions	Transformers 4.41.1, Pytorch 2.1.0

are set to 4 and 8, respectively, influencing the trade-off between computational efficiency and model stability. Furthermore, the seed is set to 42, ensuring the experiment's reproducibility.

The experiment is distributed across multiple GPUs, with a total training batch size of 32 and a total evaluation batch size of 64. This allows the experiment to leverage the parallel processing capabilities of multiple GPUs, thereby accelerating the computation. The experiment runs for 1 epoch, mitigating overfitting and providing trustworthy results. Computational resources include 16 A100 40 GB GPUs, offering substantial memory and computational power.

The framework versions used for the experiment include Transformers 4.41.1 and PyTorch 2.1.0, ensuring that the experiment is conducted using the latest versions of these libraries.

5.2. Experimental results

In this section, we present the results of our experiments using the proposed SecureQwen model for multi-class classification. We trained the model on a massive dataset of over 157k samples for a single epoch and evaluated its performance using various metrics.

5.2.1. Evaluation metrics

In the context of vulnerability detection in software, evaluating the performance of classification models is crucial to ensure they effectively identify security risks. As seen in Table 8, several key metrics are used for this purpose: precision, recall, F1-score, and accuracy. Precision (Eq. (15)) measures the rate of true positives within all predicted

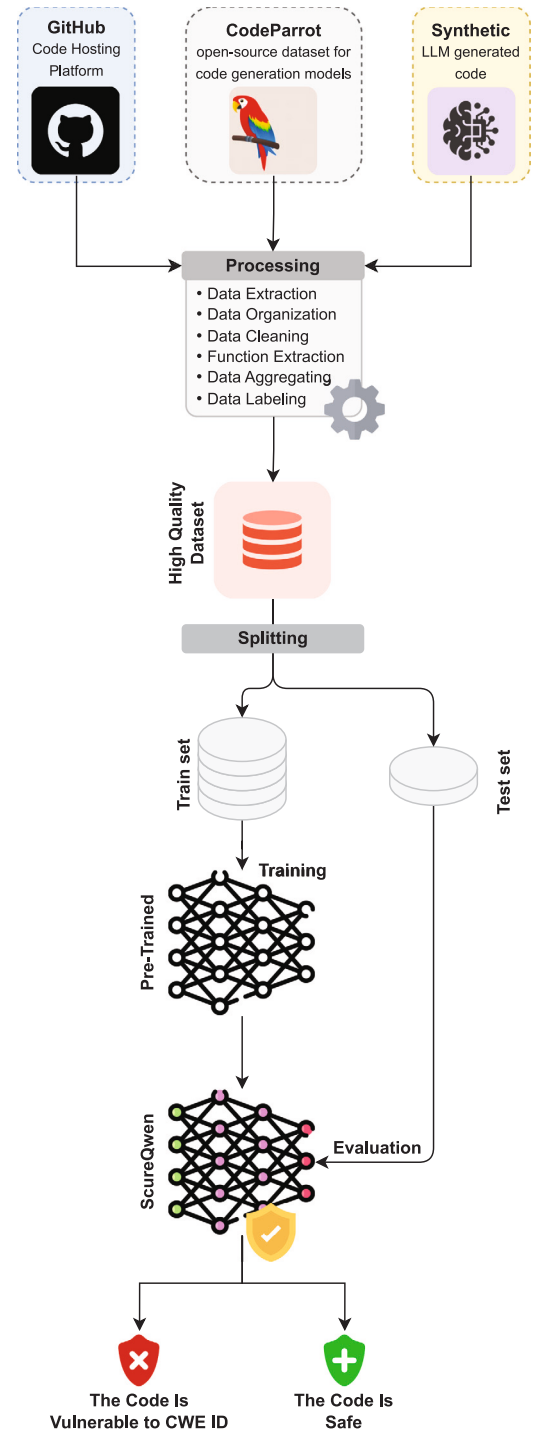


Fig. 4. Diagram illustrating the stages of data collection, preprocessing, training, and evaluation process for this experiment.

positives, indicating how many of the vulnerabilities identified by the model are actual vulnerabilities. This metric is essential for understanding the trustworthiness of the model's positive classifications and the rate of false alarms produced. Recall (Eq. (16)), or sensitivity, assesses the model's ability to correctly identify vulnerabilities out of the total actual vulnerabilities, reflecting its effectiveness in capturing all relevant security threats. The F1-score (Eq. (17)), the harmonic mean of precision and recall, provides a balanced measure that accounts for both metrics, highlighting the model's overall robustness. Ideally, a model for vulnerability detection should achieve high precision and

Table 8

Performance evaluation of SecureQwen: Function-level classification report on unseen data.

	Precision	Recall	F1-Score	Support
CWE-20	0.99	0.99	0.99	977
CWE-22	0.96	0.99	0.98	1018
CWE-259	0.98	0.97	0.97	1025
CWE-295	0.76	0.94	0.84	110
CWE-319	0.99	1.00	0.99	869
CWE-327	0.97	0.96	0.96	1048
CWE-330	0.97	0.94	0.96	1029
CWE-377	0.92	0.82	0.87	740
CWE-400	0.93	0.97	0.95	993
CWE-502	0.97	0.95	0.96	973
CWE-605	0.96	0.99	0.97	973
CWE-703	0.96	0.90	0.93	1035
CWE-78	0.90	0.98	0.94	1009
CWE-89	0.96	0.97	0.97	1034
SAFE	0.94	0.94	0.94	2898
Accuracy	0.95			
Macro Avg	0.94	0.95	0.95	15,731
Weighted Avg	0.95	0.95	0.95	15,731

recall, resulting in an F1 score close to 1, which indicates minimal false positives and false negatives. Additionally, accuracy (Eq. (18)) represents the fraction of correct predictions out of all predictions, offering a general measure of the model's overall performance. To thoroughly evaluate the model, We utilized these metrics and plotted the confusion matrix for all 15 classes (see Fig. 5), providing a detailed visualization of its classification capabilities in detecting software vulnerabilities.

$$\text{Precision} = \frac{TP}{TP + FP} \quad (15)$$

$$\text{Recall} = \frac{TP}{TP + FN} \quad (16)$$

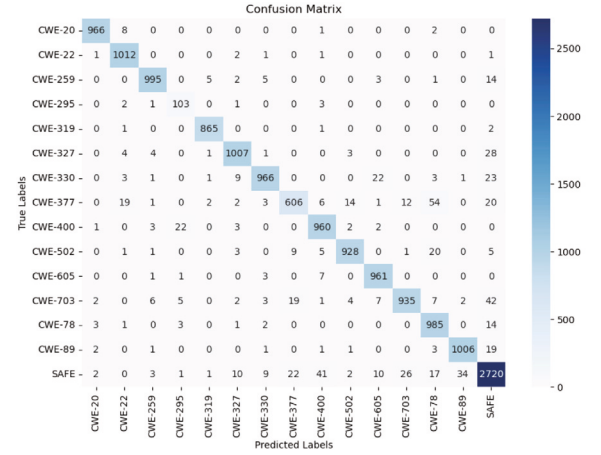
$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}} \quad (17)$$

$$\text{Accuracy} = \frac{TP + TN}{TP + FP + TN + FN} \quad (18)$$

5.2.2. Results

We present the evaluation results of our SecureQwen model on the task of multi-class vulnerability detection. As shown in Fig. 5, the confusion matrix reveals the model's impressive performance in identifying software vulnerabilities across all 15 classes. The matrix's diagonal elements, representing the number of true positives, are predominantly dark, indicating that the model accurately predicted the true labels for most classes. This suggests that the model is highly effective in detecting vulnerabilities, with a high degree of precision and recall. In particular, the model demonstrates excellent performance in detecting high-severity vulnerabilities, such as CWE-20 and CWE-78, which attackers commonly exploit. The model's ability to accurately identify these vulnerabilities is critical in preventing potential security breaches. Furthermore, the model's high recall for classes such as CWE-330 and CWE-400 indicates its effectiveness in detecting cryptographic errors and input validation vulnerabilities, respectively. While the model's performance is consistent across classes, some minor variations are observed. For instance, the model's precision for class CWE-295 is slightly lower than for other classes, which may be attributed to the relatively limited number of training samples for this class. Nevertheless, the model's overall performance is robust, with a high F1-score and accuracy, demonstrating its potential to identify security risks in software systems. The classification report in Table 8 provides a detailed breakdown of the model's performance for each class, highlighting its strengths in detecting software vulnerabilities.

In addition, we also observed stable and convergent training dynamics (Fig. 6). The training loss decreased steadily throughout the

**Fig. 5.** Function-level confusion matrix of *SecureQwen* on unseen data.

training process, indicating that the model could effectively minimize the loss function and learn meaningful representations of the input data. Furthermore, the gradient norm remained relatively stable and small, suggesting that the model's parameters were being updated in a controlled and gradual manner. Notably, we trained the model for only a single epoch, which helped to mitigate the risk of overfitting and allowed the model to generalize well to unseen data. This efficient training regime, combined with the stable training dynamics and convergent loss curve, provides further evidence of the model's robustness and ability to detect software vulnerabilities effectively.

5.3. Supplementary experiments

In addition to the primary experiments presented in this study, we conducted several supplementary experiments to explore the capabilities of SecureQwen. These experiments provide additional insights into whether the model can handle source code level classification and leverage the model's large input size while preserving the same performance.

To tackle the source code level, we relied solely on LLM-generated data using GPT-4o,⁴ a powerful large language model released by OpenAI, ensuring high-quality data. More details on the resulting dataset can be found in Tables 10 & 11. Regarding the training process and parameters, we preserved the same setup as the previous experiment (explained in Section Experimental Setup) with slight modifications. Specifically, we adjusted the number of labels to 13 and the batch size (total training batch size to 16 instead of 32 and total evaluation batch size to 32 instead of 64) to accommodate our hardware constraints.

The training loss exhibited a consistent downward trend, devoid of anomalous patterns. Using a single epoch with distinct samples supports the model's generalization ability, indicating it is not susceptible to overfitting. Furthermore, the gradient norm remained relatively stable, indicating that the parameter updates were regulated and exhibited no signs of divergence.

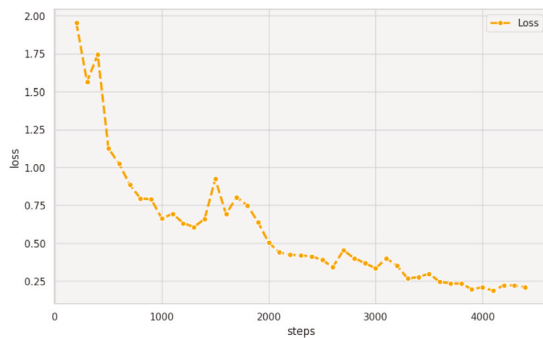
The confusion matrix (Fig. 7) and the classification report (Table 12) indicate that the model achieved high accuracy in classifying most vulnerabilities. However, there is a slight misclassification between CWE-77 and CWE-78, which their hierarchical relationship can explain. As CWE-77 is a parent of CWE-78 and shares similar criteria, this overlap leads to confusion in the model's classification. These experiments demonstrate that SecureQwen can effectively perform source code-level classification, leveraging its large input size to gain deeper insights into code dependencies and the used libraries (see Fig. 8).

⁴ <https://openai.com/index/hello-gpt-4o/>.

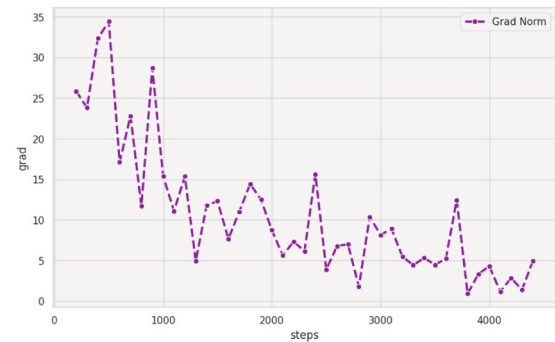
Table 9

Comparison of different vulnerability detection approaches.

Characteristics										Metrics			
Name	Year	Dataset	Lang	Data	Labeling	Multi-class	Samples	Classification method	Method	Acc	Pre	Rec	F1
Li et al. (2018)	2018	SARD	C/C++	Real& Synthetic	Patches & manual	No	10.6k programs	Binary classes	BLSTM	–	86%–91%	–	85%–95%
Russell et al. (2018)	2018	Draper	C/C++	Real	Static analyzers	Yes	1.27M functions	CWE	CNN	–	94%	–	84%
Zhou et al. (2019)	2019	Devign	C	Real	Binary Manual labeling	No	26k functions	Binary classes	GNN	72%	–	–	73%
Zou et al. (2019)	2029	SARD	C/C++	Real& Synthetic	Patches & manual	Yes	33.4k programs	CWEs	BLSTM	–	–	–	94%
Li et al. (2021)	2021	SARD	C/C++	Real & Synthetic	Patches & manual	No	15.5k programs	Binary classes	BGRU	98%	90%	–	92%
Buratti et al. (2020)	2020	Devign	C	Real	Binary Manual labeling	No	26k functions	Binary classes	Transformer model	65.45%	–	–	–
Hanif and Maffeis (2022)	2021	D2A	C/C++	Real	Vulnerability fixing commits	Yes	1.3M function and code	Categories based on static analyzer	Transformer model + MLP	62.3%	–	–	–
Hanif and Maffeis (2022)	2021	D2A	C/C++	Real	Vulnerability fixing commits	Yes	1.3M function and code	Categories based on static analyzer	Transformer model + CNN	60.7%	–	–	–
Phan et al. (2021)	2021	Devign	C	Real	Binary Manual labeling	No	26k functions	Binary classes	Transformer model	66.62%	–	–	–
Wartschinski et al. (2022)	2022	VUDENC	Python	Real	Patches	Yes	1009 commits	Binary classes	LSTM	92%–98%	82%–96%	78%–87%	80%–90%
Hu et al. (2023)	2023	BigVul	C/C++	Real	Vulnerability-fixing commits	Yes	264k programs	CVEs/CWEs	GNN	60%	–	–	–
Ferrag et al. (2024)	2024	FormAI	C/C++	Real & Synthetic	Formal verification	Yes	750k functions	CWEs	Transformer model	92%	27%–100%	14%–100%	22%–100%
Farasat and Posegga (2024)	2024	VUDENC	Python	Real	Patches	Yes	1009 commits	7 Vulnerabilities	BiLSTM	98.6%	96.2%	93.3%	94.7%
Ehrenberg et al. (2024)	2024	PyVulDet-NER	Python	Real	Vulnerability-fixing commits	Yes	298K example snippets	6 CWEs	Transformer model with NER	97%	91%–92%	91%–92%	91%–92%
Shestov et al. (2024)	2024	CVEfixes	Java	Real	Vulnerability-fixing commits	No	23.5k functions	Binary classes	Transformer model	–	–	–	71%
SecureQwen	2024	PythonVulnDB	Python	Synthetic	LLM (Gpt-4o)	Yes	165.9k programs	CWEs	Transformer model	92%	66%–99%	34%–99%	45%–99%
				Real& Synthetic	Static analysis	Yes	157.3k functions	CWEs	Transformer model	95%	90%–99%	82%–100%	84%–99%



a. Training Loss



b. Training Gradient Norm

Fig. 6. Training curves for function-level classification.

Table 10
Source code level training dataset.

Label	Tokens	Mean	Std	Min	Max	Count
SAFE	49,717k	949.96	250.44	10	2048	52,336
CWE-259	7929k	520.63	295.83	11	1926	15,231
CWE-89	7820k	793.52	187.72	284	2044	9,856
CWE-78	7593k	658.38	187.21	157	2048	11,533
CWE-502	7171k	760.26	181.78	214	1999	9,433
CWE-328	5347k	786.62	197.28	242	2048	6,798
CWE-611	5157k	730.26	195.99	216	1879	7,063
CWE-327	4919k	816.67	196.18	200	2048	6,024
CWE-77	4678k	665.04	187.10	22	1721	7,035
CWE-377	4637k	736.79	192.13	279	1963	6,294
CWE-22	4564k	716.16	194.25	90	1875	6,374
CWE-605	4528k	765.83	193.35	265	1798	5,913
CWE-319	4306k	793.42	200.54	168	2048	5,428

Table 11
Source code level test dataset.

Label	Tokens	Mean	Std	Min	Max	Count
SAFE	5594k	953.75	249.25	184	2048	5866
CWE-259	887k	510.26	301.89	65	1685	1739
CWE-89	861k	802.88	191.63	308	1529	1073
CWE-78	835k	659.59	194.15	242	1704	1267
CWE-502	755k	762.65	185.34	344	1730	991
CWE-328	630k	786.09	191.60	318	1403	802
CWE-327	574k	816.80	193.19	375	1612	703
CWE-611	557k	736.14	195.58	255	1417	758
CWE-77	524k	662.50	180.54	243	1538	792
CWE-377	515k	760.89	197.48	315	1629	678
CWE-22	514k	717.45	190.00	294	1400	717
CWE-605	510k	768.01	191.81	308	1484	665
CWE-319	425k	787.82	203.19	282	1640	540

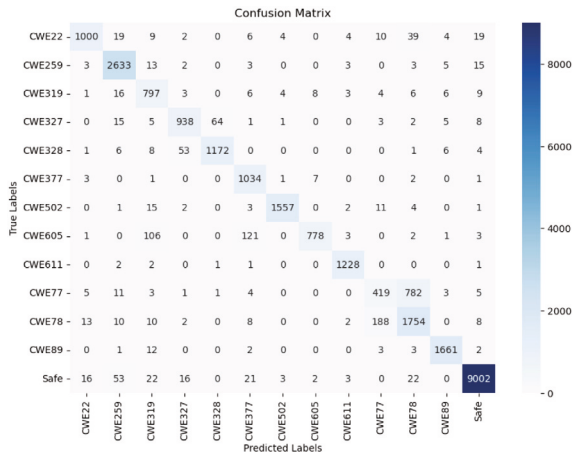


Fig. 7. Confusion matrix for source code level classification on unseen data.

5.4. Discussion

To evaluate the efficacy of SecureQwen, Table 9 presents a detailed comparison with its most closely related approaches in the field of vulnerability detection.

Russell et al. (2018) proposed a large-scale function-level vulnerability detection system using deep feature representation learning, specifically leveraging a CNN with a Random Forest classifier. Their approach, evaluated on real software packages (Draper dataset) and the NIST SATE IV benchmark, achieved a precision of 94% and an F1 score of 84%. Hanif and Maffei (2022) introduce VulBERTa, a deep-learning approach designed to detect security vulnerabilities in source code. VulBERTa leverages a RoBERTa model pre-trained with a custom tokenization pipeline on real-world C/C++ code from open-source

Table 12
Classification report for source code level classification on unseen data.

	Precision	Recall	F1-Score	Support
CWE22	0.96	0.90	0.93	1116
CWE259	0.95	0.98	0.97	2680
CWE319	0.79	0.92	0.85	863
CWE327	0.92	0.90	0.91	1042
CWE328	0.95	0.94	0.94	1251
CWE377	0.85	0.99	0.92	1049
CWE502	0.99	0.98	0.98	1596
CWE605	0.98	0.77	0.86	1015
CWE611	0.98	0.99	0.99	1235
CWE77	0.66	0.34	0.45	1234
CWE78	0.67	0.88	0.76	1995
CWE89	0.98	0.99	0.98	1684
Safe	0.99	0.98	0.99	9160
Accuracy	0.92			
Macro Avg	0.90	0.89	0.89	25,920
Weighted Avg	0.93	0.92	0.92	25,920

projects, effectively capturing deep syntax and semantic representations. Evaluations on the CodeXGLUE benchmark (Lu et al., 2021) show VulBERTa-MLP achieving 64.75% accuracy, ranking 3rd despite having significantly fewer parameters and less training data than the top models. On the D2 A benchmark (Zheng et al., 2021), VulBERTa-MLP achieved the highest accuracy at 62.30%, with VulBERTa-CNN closely following, outperforming larger models like C-BERT (Buratti et al., 2020).

Phan et al. (2021) proposed CoText, a pre-trained transformer-based model that effectively bridges natural language (NL) and programming language (PL) contexts, enabling tasks such as code summarization, generation, defect detection, and debugging. By training on extensive programming language corpora, CoText achieves state-of-the-art results across multiple tasks within the CodeXGLUE dataset, including a 66.62% accuracy on the Devign dataset for defect detection. The model's versatility is demonstrated through its success in both “bimodal” (NL-PL) and “unimodal” (PL-only) data training scenarios. Buratti et al. (2020) investigate the Software Naturalness hypothesis, proposing that programming languages can be analyzed using natural language processing techniques. By employing a pre-trained transformer-based language model directly on raw source code, they bypass the need for traditional Abstract Syntax Tree (AST) features and achieve a 65.45% accuracy on the Devign dataset for vulnerability identification. While this is slightly lower than the 66.62% accuracy achieved by CoText on the same dataset, it demonstrates that transformer-based models can still effectively perform code analysis tasks and automatically discover important features.

Li et al. (2018) present a system for detecting software vulnerabilities called VulDeePecker. The paper focuses on C/C++ programs and library/API function call vulnerabilities. The authors collected data from two sources maintained by the National Institute of Standards and Technology (NIST): the National Vulnerability Database (NVD)⁵ and the Software Assurance Reference Dataset (SARD) project (Black, 2017). They selected 19 popular C/C++ open-source products and collected 10.6k programs containing two types of vulnerabilities: buffer errors and resource management errors. The dataset contains 61,638 code gadgets, including 17,725 vulnerable and 43,913 non-vulnerable code gadgets. The VulDeePecker system uses a Bidirectional Long Short-Term Memory (BLSTM) neural network architecture for vulnerability detection. The input to the network is a vector representation of code gadgets, which are composed of several semantically related lines of code. The output is a classification of whether the code gadget is vulnerable or not. The performance of VulDeePecker was evaluated through experiments, and the results showed that it can achieve a very

⁵ <https://nvd.nist.gov>.

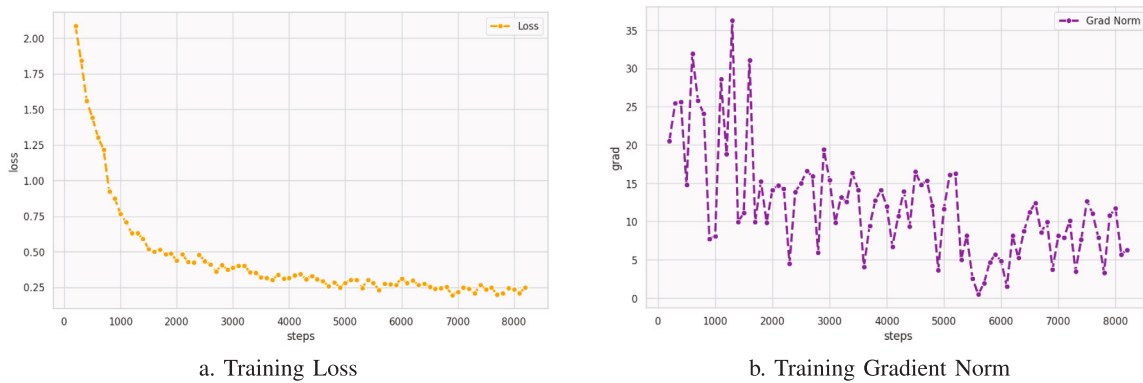


Fig. 8. Training curves for source code-level classification.

Table 13

Comparison of labeling strategies for vulnerability classification at source code and function code levels.

Labeling strategy	Source code level			Function code level		
	Accuracy (%)	Precision (%)	Recall (%)	Accuracy (%)	Precision (%)	Recall (%)
Most frequent CWE	92	90	89	95	94	95
Weighted approach	87	85	83	90	88	86
Random selection	84	82	80	88	86	84

good F1 score of around 85%–95%. However, it cannot give the type of the detected vulnerability due to the use of binary classification.

A multi-class vulnerability detection system named μ VulDeePecker was proposed by Zou et al. (2019) as an extension of their previous work, VulDeePecker. The authors collected data from the National Vulnerability Database (NVD) and the Software Assurance Reference Dataset (SARD), selecting 33.4k programs with 40 different vulnerability types, including buffer and resource management errors. μ VulDeePecker uses a Bidirectional Long Short-Term Memory (BLSTM) neural network. The system processes code gadgets through four steps: extraction, analysis to generate code attentions, extraction of attentions, and conversion into fixed-length vectors for the LSTM input. Evaluations show an F1 score of around 94%, outperforming the binary classification of VulDeePecker by identifying specific vulnerability types. However, μ VulDeePecker's coarse detection granularity requires additional inspection to locate vulnerabilities precisely, and it primarily focuses on vulnerabilities related to library/API function calls. This focus is especially significant because C/C++ are low-level languages that allow full control of memory, making vulnerabilities related to memory management common and typically hard-coded rather than arising from API calls.

SySeVR (Li et al., 2021) utilizes a BGRU model for detecting vulnerabilities in C/C++ code. This framework, known as Syntax-based, Semantics-based, and Vector Representations (SySeVR), is notable as the first systematic approach leveraging deep learning for vulnerability detection. SySeVR focuses on extracting program representations that capture both semantic and syntactic information related to vulnerabilities. The framework processes data in three stages: initially, syntax-based vulnerability candidates (SyVCs) are extracted from the training program; next, these SyVCs are transformed into semantics-based vulnerability candidates (SeVCs); and finally, SeVCs are converted into vector representations. Vulnerability detection operates at the SeVC level, encompassing multiple lines of semantically related code. Although SySeVR demonstrates significant performance improvements over state-of-the-art methods, achieving an accuracy of 98% and a recall of 92%, it has certain limitations. The detection granularity is coarse, focusing on SeVCs, and the system is confined to binary classification, lacking the capability for fine-grained vulnerability detection.

Ferrag et al. (2024) present SecureFalcon, a lightweight and innovative Large Language Model (LLM) for software vulnerability detection.

The model is derived from the Falcon-40B⁶ model and is tailored to classify software vulnerabilities. SecureFalcon is fine-tuned using a massive dataset (750k functions) gathered from various sources: the FormAI dataset, which contains C code snippets generated by a GPT model and verified using a model checker, and FalconVulnDB, an aggregated dataset of public datasets, including SySeVR, Draper VDISC, BigVul, DiverseVul, SARD Juliet, and ReVeal. SecureFalcon aims to bridge the gap between accuracy and speed. While it demonstrates good performance with 92% in multiclassification, there is some performance variability. Nonetheless, it showcases the potential of large language models in this domain.

Hovsepian et al. (2012) focus on Java vulnerabilities using a Grid Search model, achieving high precision (85%) and recall (88%). However, their work is constrained by a small dataset of 177 files from a single project, limiting its generalizability and confined to binary classification. Shestov et al. (2024) utilize a Transformer model to detect Java vulnerabilities, improving on the state-of-the-art LLM, WizardCoder (Luo et al., 2023), an enhancement of StarCoder. They adapt WizardCoder for vulnerability detection by constructing a dataset from several open-source sources: CVEfixes, a manually curated dataset, and VCMatch. This dataset contains Java functions labeled as vulnerable or non-vulnerable based on heuristics from analyzing vulnerability-fixing commits. The fine-tuned WizardCoder model demonstrates improvements in ROC AUC and F1 scores over a CodeBERT-like model, with an ROC AUC score of 69% and an F1 score of 71% on a balanced dataset without easy negative examples. The study concludes that fine-tuning large language models is effective for vulnerability detection, with the main performance bottlenecks likely related to dataset quality and the use of project-level context information.

Vudenc (Wartschinski et al., 2022) utilizes an LSTM model for detecting Python vulnerabilities. Vudenc targets seven common vulnerability types: SQL injection, cross-site scripting, command injection, cross-site request forgery, remote code execution, path disclosure, and open redirect. However, each vulnerability type is treated individually as a binary classification task, with a separate training dataset for each type. The authors used a dataset of 1009 vulnerability-fixing commits from various GitHub repositories. They propose a method for labeling code snippets as vulnerable or non-vulnerable based on

⁶ <https://huggingface.co/tiiuae/falcon-40b>.

Table 14
Comparative performance metrics on VUDENC dataset.

Vulnerabilities	Metrics	Wartschinski et al. (2022)	Farasat and Posegga (2024)	Our SecureQwen model
SQL Injection	Precision	82.2%	96.8%	97.9%
	Recall	78.0%	93.8%	94.9%
	F-Score	80.1%	95.3%	96.6%
	Accuracy	92.5%	98.2%	99.4%
XSS	Precision	91.9%	94.8%	95.9%
	Recall	80.8%	91.3%	92.5%
	F-Score	86.0%	93.0%	94.5%
	Accuracy	97.8%	98.8%	99.9%
Command injection	Precision	94.0%	97.8%	98.9%
	Recall	87.2%	95.7%	96.9%
	F-Score	90.5%	96.7%	97.7%
	Accuracy	97.8%	99.1%	99.6%
XSRF	Precision	92.9%	96.7%	97.4%
	Recall	85.4%	90.7%	91.5%
	F-Score	89.0%	93.6%	94.3%
	Accuracy	97.2%	98.3%	99.1%
Remote code execution	Precision	96.0%	97.2%	98.2%
	Recall	82.2%	95.9%	96.5%
	F-Score	88.8%	96.5%	97.8%
	Accuracy	98.1%	99.4%	99.7%
Path disclosure	Precision	92.0%	97.7%	98.7%
	Recall	84.4%	96.9%	97.5%
	F-Score	88.1%	97.3%	98.5%
	Accuracy	97.3%	99.3%	99.4%
Open redirect	Precision	91.0%	92.5%	93.5%
	Recall	83.9%	89.0%	90.2%
	F-Score	87.3%	90.7%	91.5%
	Accuracy	96.8%	97.5%	98.1%

commit contexts and filtering out irrelevant code. The code is then preprocessed and embedded using word2vec, and an LSTM network is trained to detect vulnerabilities at the level of individual tokens. The evaluation of Vudenc shows promising results, with recall ranging from 78% to 87%, precision from 82% to 96%, and F1 scores from 80% to 90%. The model can highlight specific areas in the code likely to contain vulnerabilities. However, the study has limitations, including reliance on commit contexts for labeling, which could lead to false positives or negatives if the commits are mislabeled or do not fix issues. Additionally, the use of binary classification rather than multiclass classification and the relatively small dataset for each vulnerability type are other limitations.

Ehrenberg et al. (2024) introduce a novel approach to vulnerability detection using Named Entity Recognition (NER) on a Python-based dataset collected from GitHub. Five transformer-based NER models were trained, achieving an impressive average F1-score of 91.85% across six different CWEs, outperforming previous studies in Python code vulnerability detection. The study also highlights that CodeBERT embeddings significantly surpass non-programming language-based models like RoBERTa and DistilBERT in detecting vulnerabilities at the token level. Additionally, SecureQwen is capable of detecting more than 10 CWEs and can process sequences of up to 64k tokens. In contrast, BERT-based models are limited to processing only 512 tokens, further demonstrating its superiority in large-scale vulnerability detection.

While each method has its strengths and limitations, SecureQwen's comprehensive approach stands out due to its reliance on a state-of-the-art code pretrained Transformer model and the diversity of its high-quality dataset. The dataset is well-preprocessed and represents real-life coding scenarios, combining synthetically generated data from AI models and real data scraped and collected from GitHub. Additionally, the robust labeling method, which considers a wide range of common vulnerability types, and the use of multiclass classification contribute to its robust performance across multiple metrics and labels. This makes SecureQwen the most effective among the compared methods and a reliable tool for vulnerability detection.

5.5. Evaluation of labeling strategies

In our study, we explored various strategies for labeling vulnerabilities detected at both the source code and function code levels. We evaluated three primary strategies: using the most frequent CWE label, weighted approach, and random selection. The results, presented in Table 13, show that using the most frequent CWE label consistently outperformed the other methods across both levels of code analysis. The most frequent CWE label strategy involves assigning the CWE label that appears most frequently among the detected vulnerabilities for a given code sample. This approach assumes that the most common vulnerability is the most representative of the code's security issue. The weighted approach assigns a CWE label based on a weighted scoring system, where labels are weighted according to factors such as the vulnerability's severity and the detection tool's confidence level. This method balances different aspects of detected vulnerabilities to provide a more nuanced label. Finally, random selection involves randomly choosing one of the CWE labels detected for a sample, without consideration of frequency or severity.

At the source code level, the most frequent CWE label strategy achieved the highest accuracy of 92%, with a precision of 90% and recall of 89%. The weighted approach and random selection resulted in lower accuracies of 87% and 84%, respectively. Similarly, at the function code level, the most frequent CWE label again proved to be the most effective, attaining an accuracy of 95%, precision of 94%, and recall of 95%. The other strategies showed lower performance, with the weighted approach achieving 90% accuracy and random selection yielding 88% accuracy.

5.6. Comparative performance metrics on VUDENC dataset

Table 14 presents the comparative performance metrics on the VUDENC dataset, highlighting the performance of our SecureQwen model in comparison to the models proposed by Wartschinski et al. (2022) and Farasat and Posegga (2024). Our model demonstrates superior performance across all evaluated vulnerabilities, including SQL Injection, XSS, Command Injection, XSRF, Remote Code Execution,

Path Disclosure, and Open Redirect. For instance, in detecting SQL Injection vulnerabilities, our model achieves a precision of 97.9% and an accuracy of 99.4%, surpassing Farasat and Posegga's model, which attains a precision of 96.8% and an accuracy of 98.2%. This consistent improvement in precision, recall, F-score, and accuracy across various vulnerability types underscores the effectiveness of our approach in accurately identifying security vulnerabilities in Python code. By evaluating our model on the same dataset used by existing works, we provide a fair comparison that validates the superiority of our model and its potential impact on enhancing software security.

5.7. Future works

Future research on enhancing the SecureQwen model for vulnerability detection can focus on incorporating multiple tools for labeling, such as dynamic analyzers and formal verification methods, to provide a more comprehensive dataset and recognize vulnerabilities that static analyzers might miss. Additionally, enhancing the model's explanatory capabilities is crucial; enabling it to explain why a piece of code is vulnerable, pinpoint the exact location of the vulnerability, reduce the effort required by human analysts, and suggest best practices or remediation steps will aid developers in fixing vulnerabilities and improving overall software security. Furthermore, experimenting with a smaller version of the model by reducing the number of parameters, such as using fewer decoder layers while maintaining the same architecture, can achieve similar performance levels with reduced resource consumption, leading to faster inference times and lower computational costs. Fine-tuning and testing these lighter models can help create efficient and scalable solutions for real-world applications, significantly enhancing the effectiveness, usability, and efficiency of the SecureQwen model. Another promising direction is to explore the application of Retrieval-Augmented Generation (RAG) techniques, particularly given our model's large input size (64k). By leveraging RAG, we can gather up-to-date information on vulnerabilities and mitigation strategies, enabling the model to detect and repair vulnerabilities more effectively. This integration can significantly enhance the model's capabilities to stay current with emerging threats and provide actionable insights for developers. Finally, investigating the robustness of SecureQwen against adversarial attacks, such as those using circuit-breaking techniques, can ensure the model's reliability and security in real-world deployments.

6. Conclusion

In conclusion, we proposed a novel dataset, PythonVulnDB, designed to support Python vulnerability detection. This dataset aims to fill a gap in the availability of large-scale datasets focused on Python vulnerabilities. By combining data from GitHub scraping, Codeparrot's dataset, and synthetic data generated by advanced LLMs, PythonVulnDB offers a collection of 1.875 million function-level code snippets, which can be used to train and evaluate models in the domain of Python security. We also presented the SecureQwen model, a variant of the CodeQwen1.5-7B model, demonstrating promising results in code analysis and vulnerability detection. SecureQwen leverages a decoder-only transformer architecture with enhanced capabilities for handling long contexts and efficient inference. Our findings suggest that decoder-based models can effectively generate code and perform classification tasks with minimal modifications. This dual functionality hints at the potential of such models in broader applications within the software development lifecycle, including code review, bug detection, and automated security auditing. Overall, the creation of the PythonVulnDB dataset and the development of the SecureQwen model contribute to the field of cybersecurity, providing useful tools for enhancing the security of Python applications through improved detection and classification of vulnerabilities.

CRedit authorship contribution statement

Abdechakour Mechri: Writing – review & editing, Writing – original draft, Validation, Software, Methodology, Data curation. **Mohamed Amine Ferrag:** Writing – review & editing, Writing – original draft, Supervision, Methodology, Data curation. **Merouane Debbah:** Supervision.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

The authors would like to thank the Technology Innovation Institute for providing the computing resources necessary for training the models used in this research. This work would not have been possible without the institute's support.

Data availability

Data will be made available on request.

References

- Ahmad, B., Thakur, S., Tan, B., Karri, R., Pearce, H., 2024. On hardware security bug code fixes by prompting large language models. *IEEE Trans. Inf. Forensics Secur.*
- Alammar, J., 2018. The illustrated transformer. [Online]. Available: <http://jalamar.github.io/illustrated-transformer/>.
- Alfadel, M., Costa, D.E., Shihab, E., 2023. Empirical analysis of security vulnerabilities in python packages. *Empir. Softw. Eng.* 28 (3), 59.
- Alikhanifard, P., Tsantalis, N., 2024. A novel refactoring and semantic aware abstract syntax tree differencing tool and a benchmark for evaluating the accuracy of diff tools. *ACM Trans. Softw. Eng. Methodol.*
- Antal, G., Keleti, M., Hegedüs, P., 2020. Exploring the security awareness of the python and javascript open source communities. In: *Proceedings of the 17th International Conference on Mining Software Repositories*. pp. 16–20.
- Bai, J., Bai, S., Yang, S., Wang, S., Tan, S., Wang, P., Lin, J., Zhou, C., Zhou, J., 2023a. Qwen-vl: A versatile vision-language model for understanding, localization, text reading, and beyond.
- Bai, J., Bai, S., et al., 2023b. Qwen technical report. arXiv preprint [arXiv:2309.16609](https://arxiv.org/abs/2309.16609).
- Bai, J., Men, R., Yang, H., Ren, X., Dang, K., Zhang, Y., Zhou, X., Wang, P., Tan, S., Yang, A., et al., 2022. Ofasys: A multi-modal multi-task learning system for building generalist models. arXiv preprint [arXiv:2212.04408](https://arxiv.org/abs/2212.04408).
- Bhandari, G., Naseer, A., Moonen, L., 2021. Cvefixes: automated collection of vulnerabilities and their fixes from open-source software. In: *Proceedings of the 17th International Conference on Predictive Models and Data Analytics in Software Engineering*. pp. 30–39.
- Black, P., 2017. SARD: Thousands of Reference Programs for Software Assurance. no. 5, 2017-10-31. [Online]. Available: https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=923127.
- Buratti, L., Pujar, S., Bornea, M., McCarley, S., Zheng, Y., Rossiello, G., Morari, A., Laredo, J., Thost, V., Zhuang, Y., et al., 2020. Exploring software naturalness through neural language models. arXiv preprint [arXiv:2006.12641](https://arxiv.org/abs/2006.12641).
- Cheng, X., Wang, H., Hua, J., Xu, G., Sui, Y., 2021. Deepwukong: Statically detecting software vulnerabilities using deep graph neural network. *ACM Trans. Softw. Eng. Methodol. (TOSEM)* 30 (3), 1–33.
- Chowdhery, A., Narang, S., Devlin, J., Bosma, M., Mishra, G., Roberts, A., Barham, P., Chung, H.W., Sutton, C., Gehrmann, S., et al., 2023. Palm: Scaling language modeling with pathways. *J. Mach. Learn. Res.* 24 (240), 1–113.
- Ehrenberg, M., Sarkani, S., Mazzuchi, T.A., 2024. Python source code vulnerability detection with named entity recognition. *Comput. Secur.* 140, 103802.
- Elfving, S., Uchibe, E., Doya, K., 2017. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. arXiv:1702.03118.
- Eniser, H.F., Zhang, H., David, C., Wang, M., Paulsen, B., Dodds, J., Kroening, D., 2024. Towards translating real-world code with LLMs: A study of translating to rust. arXiv preprint [arXiv:2405.11514](https://arxiv.org/abs/2405.11514).
- Farasat, T., Posegga, J., 2024. Machine learning techniques for python source code vulnerability detection. arXiv preprint [arXiv:2404.09537](https://arxiv.org/abs/2404.09537).

- Farias, B., Menezes, R., Sun, Y., Cordeiro, L.C., et al., 2024. ESBMC-python: A bounded model checker for python programs. *arXiv preprint arXiv:2407.03472*.
- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., et al., 2020. Codebert: A pre-trained model for programming and natural languages. *arXiv preprint arXiv:2002.08155*.
- Ferrag, M.A., Battah, A., Tihanyi, N., Jain, R., Maimut, D., Alwahedi, F., Lestable, T., Thandi, N.S., Mechri, A., Debbah, M., Cordeiro, L.C., 2024. SecureFalcon: Are we there yet in automated software vulnerability detection with LLMs?. *arXiv:2307.06616*.
- Finamore, A., 2024. Abstract syntax trees in python <https://pybit.es/articles/ast-intro/>. (Accessed 26 April 2024).
- Gao, Z., Wang, H., Zhou, Y., Zhu, W., Zhang, C., 2023. How far have we gone in vulnerability detection using large language models *arXiv preprint arXiv:2311.12420*.
- Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., et al., 2020. Graphcodebert: Pre-training code representations with data flow *arXiv preprint arXiv:2009.08366*.
- Hanif, H., Maffei, S., 2022. Vulberta: Simplified source code pre-training for vulnerability detection In: 2022 International Joint Conference on Neural Networks. IJCNN, IEEE, pp. 1–8.
- Hashemi Chaleshtori, F., Ray, I., 2022. Automation of vulnerability information extraction using transformer-based language models In: European Symposium on Research in Computer Security. Springer, pp. 645–665.
- Hovsepian, A., Scandariato, R., Joosen, W., Walden, J., 2012. Software vulnerability prediction using text analysis techniques In: Proceedings of the 4th International Workshop on Security Measurements and Metrics. MetriSec '12, Association for Computing Machinery, New York, NY, USA, pp. 7–10, [Online]. Available: <https://doi.org/10.1145/2372225.2372230>.
- Hu, Y., Wang, S., Li, W., Peng, J., Wu, Y., Zou, D., Jin, H., 2023. Interpreters for GNN-based vulnerability detection: Are we there yet? In: Proceedings of the 32nd ACM SIGSOFT International Symposium on Software Testing and Analysis. pp. 1407–1419.
- Jensen, R.I.T., Tawosi, V., Alamir, S., 2024. Software vulnerability and functionality assessment using llms *arXiv preprint arXiv:2403.08429*.
- Jiang, Z., Gu, J., Zhu, H., Pan, D., 2024. Pre-RMSNorm and pre-CRMSNorm transformers: equivalent and efficient pre-LN transformers *Adv. Neural Inf. Process. Syst.* 36.
- Kalliamvakou, E., Gousios, G., Blincoe, K., Singer, L., German, D.M., Damian, D., 2014. The promises and perils of mining github In: Proceedings of the 11th Working Conference on Mining Software Repositories. pp. 92–101.
- Kingma, D.P., Ba, J., 2017. Adam: A method for stochastic optimization *arXiv:1412.6980*.
- Li, J., Sangalay, A., Cheng, C., Tian, Y., Yang, J., 2024. Fine tuning large language model for secure code generation In: Proceedings of the 2024 IEEE/ACM First International Conference on AI Foundation Models and Software Engineering. pp. 86–90.
- Li, Z., Zou, D., Xu, S., Jin, H., Zhu, Y., Chen, Z., 2021. Sysevr: A framework for using deep learning to detect software vulnerabilities *IEEE Trans. Dependable Secure Comput.* 19 (4), 2244–2258.
- Li, Z., Zou, D., Xu, S., Ou, X., Jin, H., Wang, S., Deng, Z., Zhong, Y., 2018. VulDeePecker: A deep learning-based system for vulnerability detection In: Proceedings 2018 Network and Distributed System Security Symposium. Internet Society, [Online]. Available: <http://dx.doi.org/10.14722/ndss.2018.23158>.
- Lin, G., Wen, S., Han, Q.-L., Zhang, J., Xiang, Y., 2020. Software vulnerability detection using deep neural networks: a survey *Proc. IEEE* 108 (10), 1825–1848.
- Lu, S., Guo, D., Ren, S., Huang, J., Svyatkovskiy, A., Blanco, A., Clement, C., Drain, D., Jiang, D., Tang, D., et al., 2021. Codexglue: A machine learning benchmark dataset for code understanding and generation *arXiv preprint arXiv:2102.04664*.
- Lu, G., Ju, X., Chen, X., Pei, W., Cai, Z., 2024. GRACE: Empowering LLM-based software vulnerability detection with graph structure and in-context learning *J. Syst. Softw.* 212, 112031.
- Luo, Z., Xu, C., Zhao, P., Sun, Q., Geng, X., Hu, W., Tao, C., Ma, J., Lin, Q., Jiang, D., 2023. WizardCoder: Empowering code large language models with evol-instruct *arXiv:2306.08568*.
- Mamede, C., Pinconschi, E., Abreu, R., 2022. A transformer-based IDE plugin for vulnerability detection In: Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering. pp. 1–4.
- Mao, Z., Li, J., Li, M., Tei, K., 2024. Multi-role consensus through LLMs discussions for vulnerability detection *arXiv preprint arXiv:2403.14274*.
- Mathews, N.S., Brus, Y., Aafer, Y., Nagappan, M., McIntosh, S., 2024. Llbepky: Leveraging large language models for vulnerability detection *arXiv preprint arXiv:2401.01269*.
- Menezes, R.S., Aldughaim, M., Farias, B., Li, X., Manino, E., Shmarov, F., Song, K., Brau e, F., Gadelha, M.R., Tihanyi, N., et al., 2024. ESBMC v7. 4: Harnessing the power of intervals: (competition contribution) In: International Conference on Tools and Algorithms for the Construction and Analysis of Systems. Springer, pp. 376–380.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G.S., Dean, J., 2013. Distributed representations of words and phrases and their compositionality In: Burges, C., Bottou, L., Welling, M., Ghahramani, Z., Weinberger, K. (Eds.), *Advances in Neural Information Processing Systems*. Vol. 26, Curran Associates, Inc., [Online]. Available: https://proceedings.neurips.cc/paper_files/paper/2013/file/9aa42b31882ec039965f3c4923ce901b-Paper.pdf.
- Mohammed, N., Lal, A., Rastogi, A., Roy, S., Sharma, R., 2024. Enabling memory safety of c programs using LLMs *arXiv preprint arXiv:2404.01096*.
- OpenAI, 2023. ChatGPT [Online]. Available: <https://chat.openai.com/>. (Accessed 23 April 2024).
- OpenAI, et al., 2023. GPT-4 Technical Report. Tech. Rep., OpenAI, <http://dx.doi.org/10.48550/arXiv.2303.08774>, *arXiv:2303.08774* [cs.CL].
- Ozturk, O.S., Ekmekcioglu, E., Cetin, O., Arief, B., Hernandez-Castro, J., 2023. New tricks to old codes: can AI chatbots replace static code analysis tools? In: *Proceedings of the 2023 European Interdisciplinary Cybersecurity Conference*. pp. 13–18.
- Phan, L., Tran, H., Le, D., Nguyen, H., Anibal, J., Peltekian, A., Ye, Y., 2021. Cotext: Multi-task learning with code-text transformer *arXiv preprint arXiv:2105.08645*.
- Popescu, M.-C., Balas, V.E., Perescu-Popescu, L., Mastorakis, N., 2009. Multilayer perceptron and neural networks WSEAS Trans. Circuits Syst. 8 (7), 579–588.
- Rozière, B., Gehring, J., Gloeckle, F., Sootla, S., Gat, I., Tan, X.E., Adi, Y., Liu, J., Sauvestre, R., Reme, T., Rapin, J., Kozhevnikov, A., Evtimov, I., Bitton, J., Bhatt, M., Ferrer, C.C., Grattafiori, A., Xiong, W., Défossez, A., Copet, J., Azhar, F., Touvron, H., Martin, L., Usunier, N., Scialom, T., Synnaeve, G., 2024. Code llama: Open foundation models for code *arXiv:2308.12950*.
- Russell, R., Kim, L., Hamilton, L., Lazovich, T., Harer, J., Ozdemir, O., Ellingwood, P., McConley, M., 2018. Automated vulnerability detection in source code using deep representation learning In: 2018 17th IEEE International Conference on Machine Learning and Applications. ICMLA, IEEE, pp. 757–762.
- Senanayake, J., Kalutara, H., Al-Kadiri, M.O., Petrovski, A., Piras, L., 2023. Android source code vulnerability detection: a systematic literature review *ACM Comput. Surv.* 55 (9), 1–37.
- Shazeer, N., 2020. Glu variants improve transformer *arXiv preprint arXiv:2002.05202*.
- Shestov, A., Levichev, R., Mussabayev, R., Maslov, E., Cheshkov, A., Zadorozhny, P., 2024. Finetuning large language models for vulnerability detection *arXiv:2401.17010*.
- Silva, A., Fang, S., Monperrus, M., 2023. RepairLLaMA: Efficient representations and fine-tuned adapters for program repair *arXiv preprint arXiv:2312.15698*.
- Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., Liu, Y., 2021. Roformer: Enhanced transformer with rotary position embedding *arXiv preprint arXiv:2104.09864*.
- Subhan, F., Wu, X., Bo, L., Sun, X., Rahman, M., 2022. A deep learning-based approach for software vulnerability detection using code metrics *IET Softw.* 16 (5), 516–526.
- Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., et al., 2023. Llama: Open and efficient foundation language models *arXiv preprint arXiv:2302.13971*.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, Ł., Polosukhin, I., 2017. Attention is all you need *Adv. Neural Inf. Process. Syst.* 30.
- Wartschinski, L., Noller, Y., Vogel, T., Kehrer, T., Grunske, L., 2022. VUDENC: vulnerability detection with deep learning on a natural codebase for python *Inf. Softw. Technol.* 144, 106809.
- Wu, F., Zhang, N., Jha, S., McDaniel, P., Xiao, C., 2024. A new era in LLM security: Exploring security concerns in real-world LLM-based systems *arXiv preprint arXiv:2402.18649*.
- Xia, C.S., Paltenghi, M., Le Tian, J., Pradel, M., Zhang, L., 2024. Fuzz4all: Universal fuzzing with large language models In: Proceedings of the IEEE/ACM 46th International Conference on Software Engineering. pp. 1–13.
- Xie, Z., Raskar, S., Emani, M., 2022. Throughput-oriented and accuracy-aware DNN training with bfloat16 on GPU In: 2022 IEEE International Parallel and Distributed Processing Symposium Workshops. IPDPSW, pp. 1084–1087. <http://dx.doi.org/10.1109/IPDPSW55747.2022.00176>.
- Yusuf, I.N.B., Jiang, L., 2024. Your instructions are not always helpful: Assessing the efficacy of instruction fine-tuning for software vulnerability detection *arXiv preprint arXiv:2401.07466*.
- Zhang, M., Arcuri, A., Li, Y., Liu, Y., Xue, K., 2023. White-box fuzzing RPC-based APIs with EvoMaster: An industrial case study *ACM Trans. Softw. Eng. Methodol.* 32 (5), 1–38.
- Zhang, Q., Fang, C., Sun, W., Liu, Y., He, T., Hao, X., Chen, Z., 2024a. APPT: Boosting automated patch correctness prediction via fine-tuning pre-trained models *IEEE Trans. Softw. Eng.*
- Zhang, B., Sennrich, R., 2019. Root mean square layer normalization *arXiv:1910.07467*.
- Zhang, L., Zou, Q., Singhal, A., Sun, X., Liu, P., 2024b. Evaluating large language models for real-world vulnerability repair in C/C++ code In: Proceedings of the 10th ACM International Workshop on Security and Privacy Analytics. pp. 49–58.
- Zhao, W.X., Zhou, K., Li, J., Tang, T., Wang, X., Hou, Y., Min, Y., Zhang, B., Zhang, J., Dong, Z., et al., 2023. A survey of large language models *arXiv preprint arXiv:2303.18223*.

- Zheng, Y., Pujar, S., Lewis, B., Buratti, L., Epstein, E., Yang, B., Laredo, J., Morari, A., Su, Z., 2021. D2a: A dataset built for ai-based vulnerability detection methods using differential analysis In: 2021 IEEE/ACM 43rd International Conference on Software Engineering: Software Engineering in Practice. ICSE-SEIP, IEEE, pp. 111–120.
- Zhou, Y., Liu, S., Siow, J., Du, X., Liu, Y., 2019. Devign: Effective vulnerability identification by learning comprehensive program semantics via graph neural networks In: Proceedings of the 33rd International Conference on Neural Information Processing Systems. Curran Associates Inc., Red Hook, NY, USA.
- Zhou, J., Pacheco, M., Wan, Z., Xia, X., Lo, D., Wang, Y., Hassan, A.E., 2021. Finding a needle in a haystack: Automated mining of silent vulnerability fixes In: 2021 36th IEEE/ACM International Conference on Automated Software Engineering. ASE, IEEE, pp. 705–716.
- Zhou, X., Tran, D.-M., Le-Cong, T., Zhang, T., Irsan, I.C., Sumarlin, J., Le, B., Lo, D., 2024. Comparison of static application security testing tools and large language models for repo-level vulnerability detection arXiv preprint arXiv:2407.16235.
- Zou, D., Wang, S., Xu, S., Li, Z., Jin, H., 2019. VulDeePecker: A deep learning-based system for multiclass vulnerability detection IEEE Trans. Dependable Secure Comput. 18 (5), 2224–2236.



Abdechakour Mechri Engineering and Master's student in Computer Science, specializing in AI and Data Science at École Supérieure d'Informatique, Sidi Bel Abbès, Algeria. Currently interning at the Technology Innovation Institute (TII). Passionate about exploring innovative solutions and new technologies. He is particularly interested in applying AI and machine learning to enhance cybersecurity.



Mohamed Amine Ferrag earned his Bachelor's, Master's, Ph.D., and Habilitation degrees in Computer Science from Badji Mokhtar—Annaba University in Annaba, Algeria, completing his studies in 2008, 2010, 2014, and 2019, respectively. He served as an Associate Professor in the Department of Computer Science at Guelma University, Algeria, from 2014 until 2022. Concurrently, from 2019 to 2022, he held the position of Senior Researcher at the NAU-Lincoln Joint Research Center of Intelligent Engineering, based at Nanjing Agricultural University in China. As of 2022, Dr. Ferrag is the Lead Researcher at the Artificial Intelligence & Digital Science Research Center at the Technology Innovation Institute in Abu Dhabi, United Arab Emirates. Dr. Ferrag's research primarily focuses on a spectrum of topics within the cybersecurity domain, including wireless network security, network coding security, applied cryptography, blockchain technology, generative AI, software security, and the application of AI in cybersecurity. His scholarly output includes over 140 papers published in international journals and conference proceedings. Dr. Ferrag has spearheaded numerous projects in research and development, fostering collaborative ties with academic institutions in the UK, Australia, USA, Canada, and China. His contributions to the field include the creation of three cybersecurity datasets, namely, Edge-IIoT dataset, FormAI dataset, and CyberMetric dataset, which have become essential resources for AI researchers worldwide. His academic contributions have been recognized with the 2021 IEEE TEM Best Paper Award, the 2022 Scopus Algeria Award, and many best paper conference awards. He has consistently been named on Stanford University's list of the world's top 2% of scientists four times from 2020 through 2023. Dr. Ferrag also contributes to the academic community as an associate editor for prestigious journals, such as the IEEE Internet of Things Journal and ICT Express (Elsevier). In addition to his research and editorial roles, Dr. Ferrag is a Senior Member of the Institute of Electrical and Electronics Engineers (IEEE).



Mérouane Debbah is Professor at Khalifa University of Science and Technology in Abu Dhabi. He received the M.Sc. and Ph. D. degrees from the Ecole Normale Supérieure Paris-Saclay, France. He was with Motorola Labs, Saclay, France, from 1999 to 2002, and then with the Vienna Research Center for Telecommunications, Vienna, Austria, until 2003. From 2003 to 2007, he was an Assistant Professor with the Mobile Communications Department, Institut Eurecom, Sophia Antipolis, France. Since 2007, he is Full Professor at CentraleSupélec, Gif-sur-Yvette, France. From 2007 to 2014, he was the Director of the Alcatel-Lucent Chair on Flexible Radio. From 2014 to 2021, he was Vice-President of the Huawei France Research Center. He was jointly the director of the Mathematical and Algorithmic Sciences Lab as well as the director of the Lagrange Mathematical and Computing Research Center. From 2021 to 2023, he was Chief Researcher at the Technology Innovation Institute and leading the AI & Digital Science Research centers at the Technology Innovation Institute. He was also Adjunct Professor with the Department of Machine Learning at the Mohamed Bin Zayed University of Artificial Intelligence in Abu Dhabi. Since 2023, he is a Professor at Khalifa University of Science and Technology in Abu Dhabi and founding director of the 6G center. He has managed 8 EU projects and more than 24 national and international projects. His research interests lie in fundamental mathematics, algorithms, statistics, information, and communication sciences research. He holds more than 50 patents. He is an IEEE Fellow, a WWRF Fellow, a Eurasp Fellow, an AIAA Fellow, an Institut Louis Bachelier Fellow and a Membre émérite SEE. He was a recipient of the ERC Grant MORE (Advanced Mathematical Tools for Complex Network Engineering) from 2012 to 2017. He was a recipient of the Mario Boella Award in 2005, the IEEE Glavieux Prize Award in 2011, the Qualcomm Innovation Prize Award in 2012, the 2019 IEEE Radio Communications Committee Technical Recognition Award and the 2020 SEE Blondel Medal. He received more than 40 best paper awards, among which the 2007 IEEE GLOBECOM Best Paper Award, the Wi-Opt 2009 Best Paper Award, the 2010 Newcom++ Best Paper Award, the WUN CogCom Best Paper 2012 and 2013 Award, the 2014 WCNC Best Paper Award, the 2015 ICC Best Paper Award, the 2015 IEEE Communications Society Leonard G. Abraham Prize, the 2015 IEEE Communications Society Fred W. Ellersick Prize, the 2016 IEEE Communications Society Best Tutorial Paper Award, the 2016 European Wireless Best Paper Award, the 2017 Eurasp Best Paper Award, the 2018 IEEE Marconi Prize Paper Award, the 2019 IEEE Communications Society Young Author Best Paper Award, the 2021 Eurasp Best Paper Award, the 2021 IEEE Marconi Prize Paper Award, the 2022 IEEE Communications Society Outstanding Paper Award, the 2022 ICC Best paper Award, the 2022 IEEE GLOBECOM Best Paper Award, 2022 IEEE TAOS TC Best GCSN Paper Award, the 2022 IEEE International Conference on Metaverse Best Paper Award, the 2023 IEEE Communications Society Fred W. Ellersick Prize, the 2023 ICC best paper award, the 2023 Best ICEAA-IEEE APWC Paper Award, the 2023 IEEE ICCT Best Paper Award, the 2023 IEEE Communications Society Award for Advances in Communication, the 2024 IEEE ICC Best Paper Award, the 2024 IEEE Communications Society Leonard G. Abraham Prize, the 2024 IEEE Communications Society Best Tutorial Paper Award as well as the Valuetools 2007, Valuetools 2008, CrownCom 2009, Valuetools 2012, SAM 2014, and 2017 IEEE Sweden VT-COM-IT Joint Chapter best student paper awards. He is an Associate Editor-in-Chief of the journal Random Matrix: Theory and Applications. He was an Associate Area Editor and Senior Area Editor of the IEEE TRANSACTIONS ON SIGNAL PROCESSING from 2011 to 2013 and from 2013 to 2014, respectively. From 2021 to 2022, he served as an IEEE Signal Processing Society Distinguished Industry Speaker. He is a member of the Marconi Prize Selection Advisory Committee.